

Svaya Collaborative Robots

SR-L6, SR-L10, SR-L12, SR-L16

API Manual v2.1.1



Table of Contents

Table of Contents	2
1 Svaya API.....	6
1.1 Introduction	6
1.2 Getting started with the Svaya Robot.....	6
1.3 Setting communication with the Robot	7
1.4 Robot IP Ping through CMD/Power Shell.....	9
1.5 Robot IP Ping through Py-Charm	10
1.6 Creating a New Program in Py-Charm IDE.....	12
2 Installing recommended libraries and packages.....	16
3 Svaya API Installation.....	20
4 API Commands.....	22
4.1 System Initialization	22
4.1.1 initialize().....	22
4.1.2 enableRobot().....	22
4.1.3 disableRobot().....	23
4.1.4 error_callback().....	23
4.2 Developer mode	25
4.2.1 enableDeveloperMode().....	25
4.2.2 disableDeveloperMode().....	25
4.2.3 getDeveloperModeStatus().....	25
4.2.4 sendTorqueCommand().....	26
4.2.5 getGravitytorque()	27
4.3 Mode Switching.....	29
4.3.1 switchToReal().....	29
4.3.2 switchToVirtual().....	29
4.3.3 switchToAuto().....	29
4.4 Feedback and Sensing	30
4.4.1 getJointValues().....	30
4.4.2 getJointVelocities().....	30
4.4.3 getCartPose().....	30
4.4.4 getCartPoseArray()	31
4.4.5 getQuaternion()	31
4.4.6 getQuaternionArray().....	31
4.4.7 getFTTorqueData().....	31

4.4.8	getFTForceData().....	32
4.4.9	getMotorTorqueData().....	32
4.4.10	getJTSBiasedData().....	32
4.5	Delays and Synchronization.....	33
4.5.1	waitForInput().....	33
4.5.2	waitForTime().....	33
4.6	Configuration and control.....	34
4.6.1	setDigitalOutput()	34
4.6.2	setAnalogOutput()	34
4.6.3	setToolDigitalOutput().....	34
4.6.4	setJogSpeed()	35
4.6.5	setJointJogStep().....	35
4.6.6	setCartesianJogStep()	35
4.6.7	setFTBiasState().....	36
4.6.8	setPayload().....	36
4.6.9	setRobotBaseFrame().....	37
4.6.10	resetRobotBaseFrame()	37
4.6.11	setJogUserFrame().....	37
4.6.12	resetJogUserFrame().....	38
4.6.13	setJogToolFrame().....	38
4.6.14	resetJogToolFrame().....	38
4.6.15	setJogUserAndToolFrame().....	38
4.6.16	resetJogUserAndToolFrame()	39
4.7	Motion Control.....	40
4.7.1	goToHome().....	40
4.7.2	goToHome().....	40
4.7.3	goToHome().....	40
4.7.4	goToHomeAsync().....	41
4.7.5	goToHomeAsync().....	41
4.7.6	goToHomeAsync().....	41
4.7.7	moveJoint().....	41
4.7.8	moveJointAsync()	42
4.7.9	moveJoints()	42
4.7.10	moveJoints()	42
4.7.11	moveJoints()	43
4.7.12	moveJointsAsync().....	43

4.7.13	moveJointsAsync()	44
4.7.14	moveJointsAsync()	44
4.7.15	moveToCartPose()	44
4.7.16	moveToCartPose()	45
4.7.17	moveToCartPose()	45
4.7.18	moveToCartPose()	46
4.7.19	moveToCartPose()	46
4.7.20	moveToCartPose()	46
4.7.21	moveToCartPoseAsync()	47
4.7.22	moveToCartPoseAsync()	47
4.7.23	moveToCartPoseAsync()	48
4.7.24	moveToCartPoseAsync()	48
4.7.25	moveToCartPoseAsync()	48
4.7.26	moveToCartPoseAsync()	49
4.7.27	moveTrajectory()	49
4.7.28	moveTrajectory()	50
4.7.29	moveTrajectoryAsync()	50
4.7.30	moveTrajectoryAsync()	51
4.7.31	moveTrajectoryInCircle()	51
4.7.32	moveTrajectoryInCircle()	52
4.7.33	moveTrajectoryInCircleAsync()	53
4.7.34	moveTrajectoryInCircleAsync()	53
4.8	Basic status and I/O	55
4.8.1	getState()	55
4.8.2	getGreenButton()	55
4.8.3	getRedButton()	55
4.8.4	getJointErrorsDesc()	55
4.8.5	getDigitalInput()	55
4.8.6	getDigitalInputs()	56
4.8.7	getDigitalOutput()	56
4.8.8	getDigitalOutputs()	56
4.8.9	getAnalogInput()	57
4.8.10	getAnalogInputs()	57
4.8.11	getAnalogOutput()	57
4.8.12	getAnalogOutputs()	58
4.8.13	getToolDigitalInput()	58

4.8.14	getToolDigitalInputs().....	58
4.8.15	getToolDigitalOutput()	59
4.8.16	getToolDigitalOutputs().....	59
4.8.17	getToolAnalogInput()	60
4.9	Jogging.....	61
4.9.1	jog().....	61
4.9.2	jogForTime().....	62
4.9.3	jogAlong_X()	62
4.9.4	jogAlong_Y()	63
4.9.5	jogAlong_Z()	63
4.9.6	rotateAlong_X().....	63
4.9.7	rotateAlong_Y().....	64
4.9.8	rotateAlong_Z().....	64
4.9.9	jointStepJog().....	64
4.9.10	stepJogAlong_X()	65
4.9.11	stepJogAlong_Y()	65
4.9.12	stepJogAlong_Z()	65
4.9.13	stepRotateAlong_X().....	65
4.9.14	stepRotateAlong_Y().....	65
4.9.15	stepRotateAlong_Z().....	65
4.10	Emergency and stop function.....	67
4.10.1	stopRobot()	67
4.10.2	stopJointJogging().....	67
4.10.3	stopCartesianJogging()	67
5	Example Programs.....	68
5.1	Example 1: Robot Basic Functions.....	68
5.2	Example 2: Control the Robot using an X-box controller.....	71
5.3	Example 3: Record Robot joint and sensor data.....	74
5.4	Example 4: Handling collision fault condition	76

1 Svaya API

1.1 Introduction

The Svaya Robot API is a powerful and intuitive tool designed to provide seamless control over Svaya's robotic systems. Focused on ease of use and flexibility, this API allows developers to program and automate a variety of robotic tasks using Python, leveraging its high-level capabilities for real-time applications. Currently, the API is fully integrated with PyCharm, making it easily accessible to developers using one of the most popular Python IDEs. However, the Svaya API can be used in any Python environment, not just PyCharm. If you are not using PyCharm, we recommend setting up a virtual environment to manage the API and its dependencies more easily.

Whether you are a novice or an experienced roboticist, the Svaya API provides the essential building blocks for efficient robot control. The API supports a wide range of operations from low-level joint movements to high-level automation processes enabling the creation of custom solutions in fields such as manufacturing, automation, research, and more.

Core Capabilities of Svaya API

- Joint and cartesian motion control
- Gripper and end-effector control
- Robot motion via Trajectory
- Digital and analog IO
- Real-Time feedback
- Robot Control through External devices
- Custom Python integrations

Example Applications

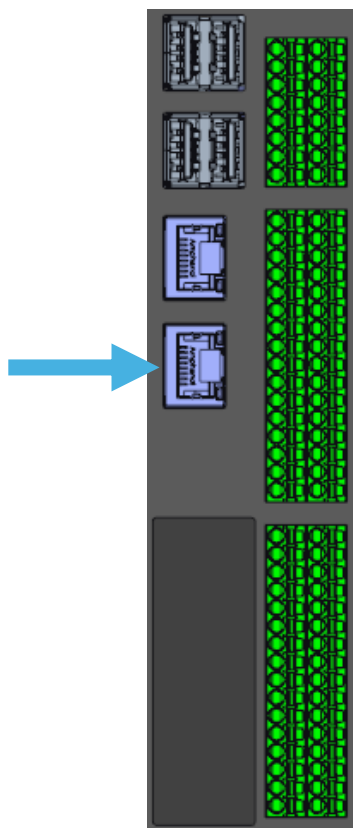
- Researchers can use the API to design and test new algorithms for robotic control, machine learning integration, or advanced path planning.
- Combined with input devices (joysticks, game controllers) via Python libraries, the API allows for interactive control, making it ideal for teleoperation or collaborative robot systems.
- Python's extensive data libraries enable the integration of robotics with machine learning, computer vision, and sensor fusion systems for intelligent automation.
- Robots can be commanded through voice recognition software and generative AI.

1.2 Getting started with the Svaya Robot

The following are the steps to be taken to make communication possible for the user's device to the robot for API programming.

Step 1: The user should connect the Ethernet cable to the correct port on the robot's controller box. The correct port is the "Ethernet 2 - Static IP" port. This port is not intended for an internet connection; it can only be used for Ethernet-based field devices and the Svaya API physical interface.

Step 2: Connect the Ethernet to a device where API must run. Configure the IP address of the port to which the Ethernet cable from the robot will be connected



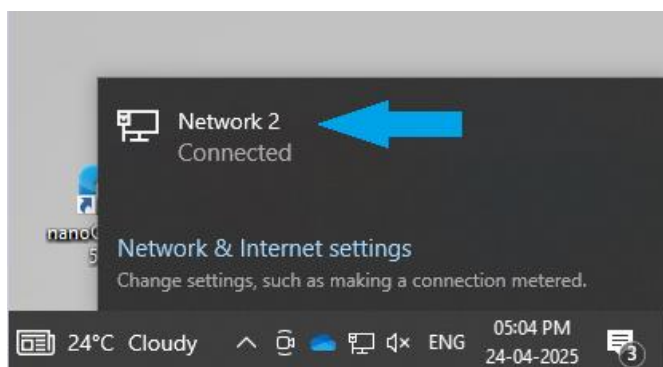
Note: Ensure that the static IP address of the Ethernet port on the user's device is within the same gateway range as the robot. The robot's Ethernet static IP is 192.168.1.10, so the user's device can have an IP address in the range of 192.168.1.11 to 192.168.1.255.

1.3 Setting communication with the Robot

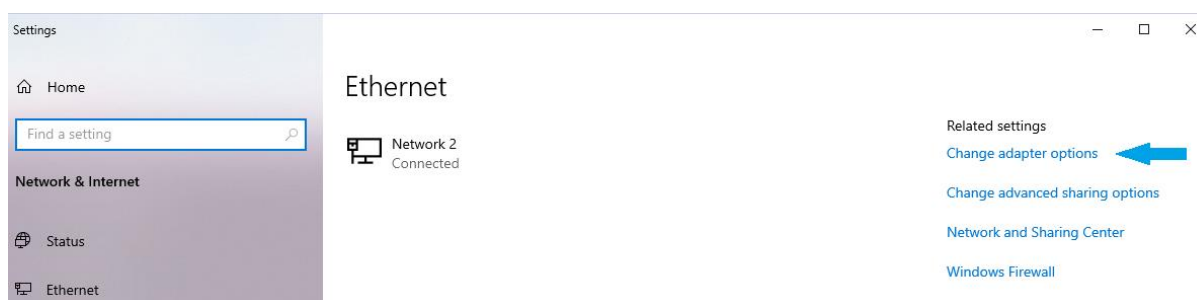
By default, the static IP address of the robot is 192.168.1.10. To set a PC to work with static IP address, follow the instructions below.

Step 1: In the right corner of the Taskbar, left click with mouse on the network connection indicator.

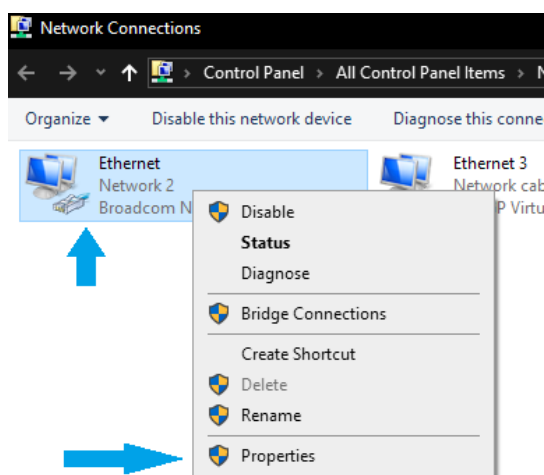
Step 2: Select the active Ethernet connection next to the network connection indicator. The connection name can differ, depending on your network settings.



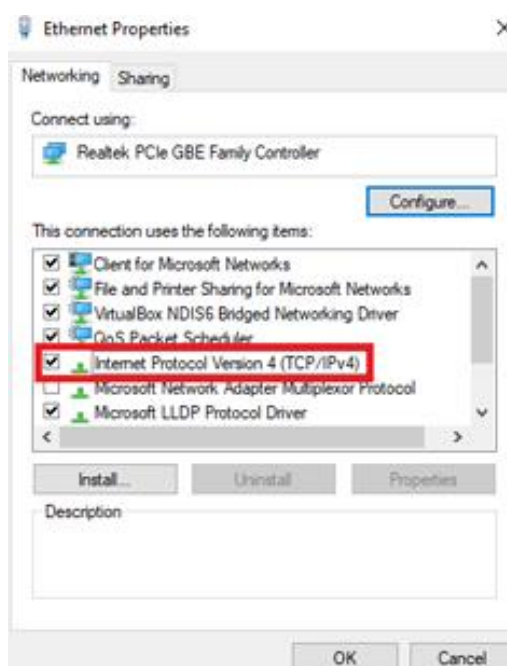
Step 3: In the displayed Ethernet settings window, click **Change adapter options**.



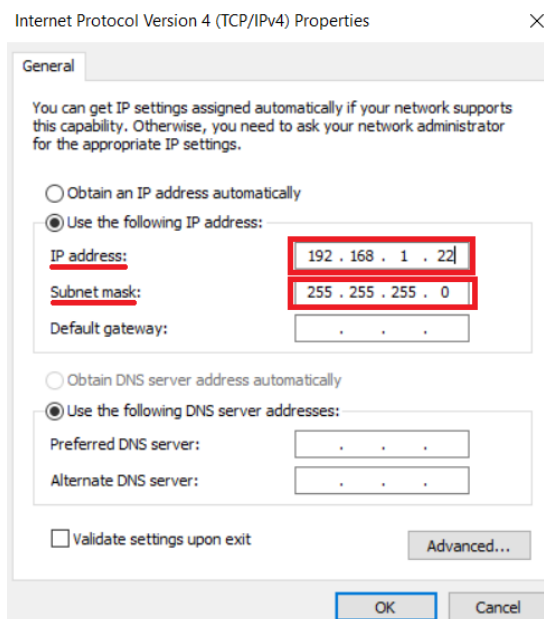
Step 4: In the displayed **Network Connections** window, right click the Ethernet icon and select **Properties**.



Step 5: On the **Networking** tab in the displayed **Ethernet properties** window, double click **Internet Protocol Version 4 (TCP/IPv4)**.



Step 6: In the **IP Version 4 (TCP/IPv4)** window, select **Use the following IP address** and set up the **IP address** and the **Subnet Mask** fields to be in same series of the Robot's static IP address



Step 7: Click **Ok** to confirm the settings.

1.4 Robot IP Ping through CMD/Power Shell

The user can perform a ping test using either Command Prompt or Windows PowerShell on a Windows system. They only need to know the robot's IP address, which can be found on the log page in the UI.

Note: The IP gateway for both the robots and the computer should be the same. For example, if the robot's IP address is 192.168.1.10, then the PC should have an IP address within the same subnet, such as 192.168.1.15, 192.168.1.100, or 192.168.1.35. The key is that both devices share the same network prefix (e.g., 192.168.1.x) but have different individual addresses.

```

Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Senthil>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 1ms, Average = 0ms

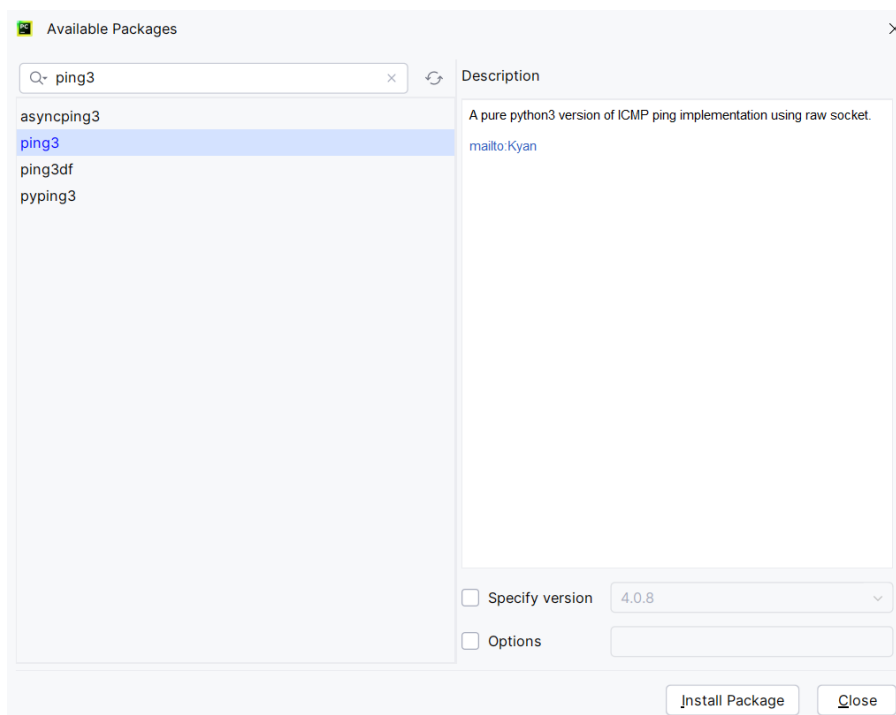
C:\Users\Senthil>

```

1.5 Robot IP Ping through Py-Charm

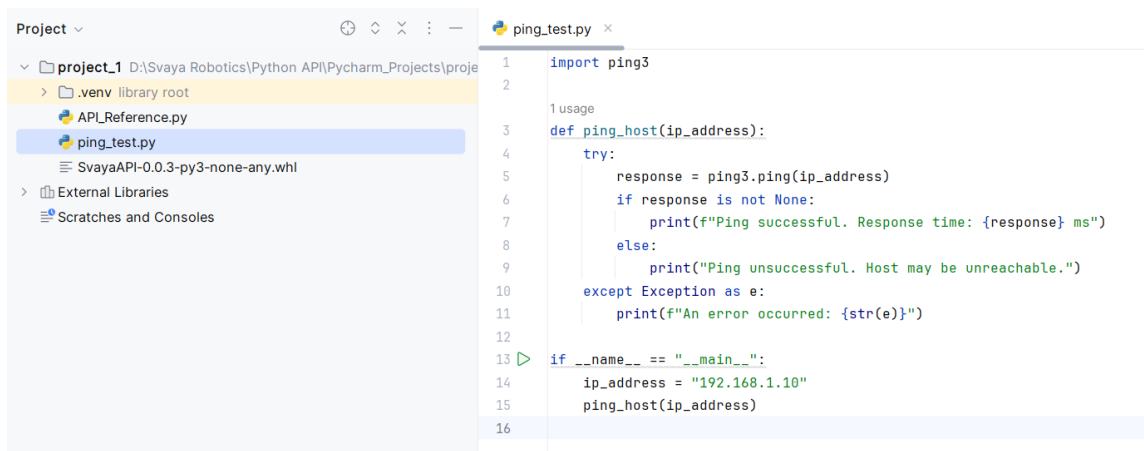
If the user is not familiar with how to ping another device from the PC, they can follow the steps below:

- 1) **Change the PC's IP Address:** Set your PC's IP address to be within the same subnet as the robot. The robot's Ethernet port uses a static IP address of 192.168.1.10. For example, you can set your PC's IP to something like 192.168.1.20.
- 2) **Install the ping3 Package in PyCharm**
 - a) Open PyCharm and go to Settings.
 - b) Navigate to Project: [Your Project Name] > Python Interpreter.
 - c) Click the “+” icon to open the available packages dialog.
 - d) In the search bar at the top, type ping3 and select version 4.0.8 or above.
 - e) Click the Install button to install the package.



3) Update the Robot's IP Address in the Script

- In the PyCharm **Project** window, open the file named ping_test.py.
- Locate **line 14** and update the IP address to match the robot's IP (e.g., 192.168.1.10).



4) Run the Program and Check the Results

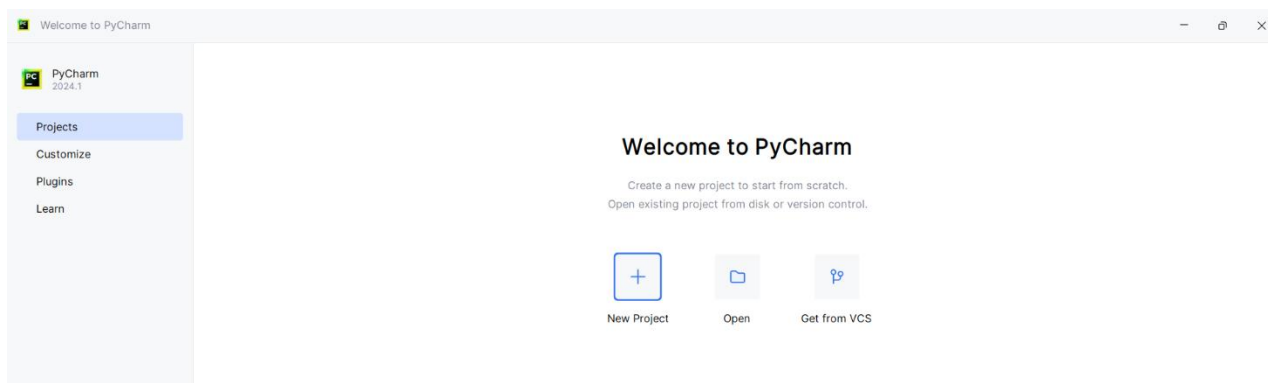
- Run the ping_test.py script.
- The results will be displayed at the bottom of the PyCharm interface (in the Run or Terminal window).
- If the ping is successful, you can proceed with the next steps.
- If the ping fails, check for possible issues such as incorrect IP settings, cable connections, or firewall restrictions and resolve them before continuing.



The following chapter will explain how to set up the PyCharm for Svaya API.

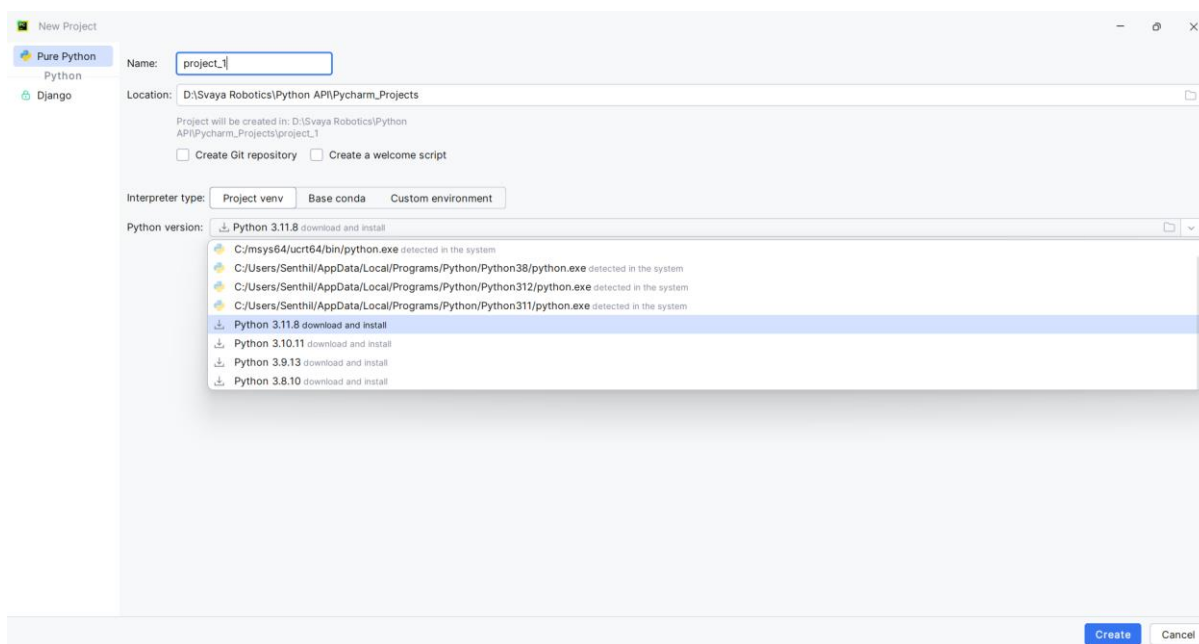
1.6 Creating a New Program in Py-Charm IDE

Step 1: Open the Python IDE - PyCharm and create a new project.

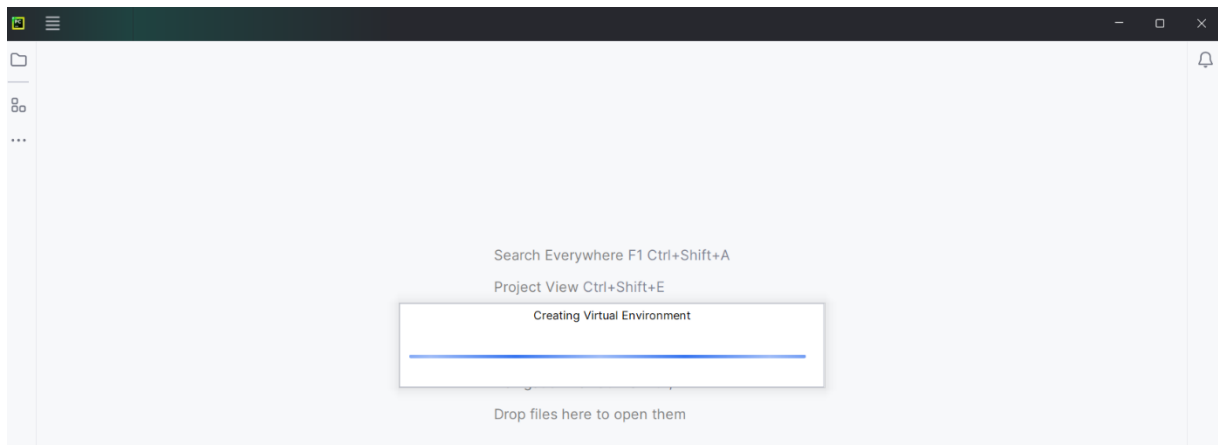


Step 2: Create a New Project in PyCharm

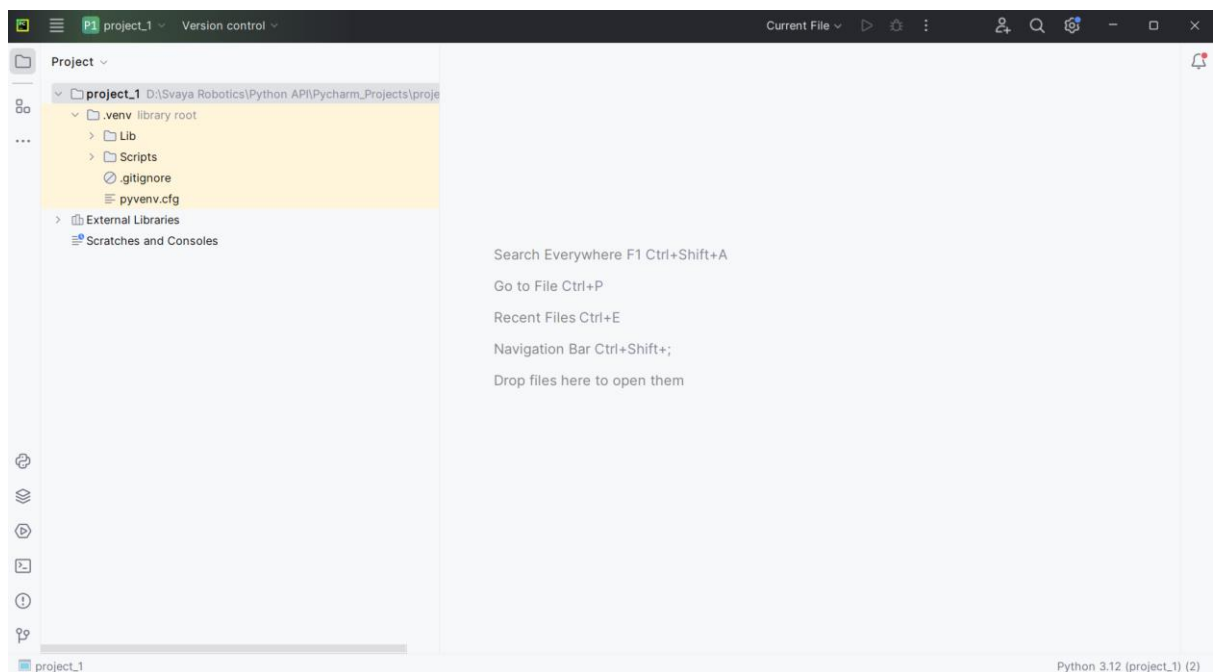
- a) Enter a name for your project.
- b) Select the desired Python interpreter. It is recommended to use **Python version 3.8 or higher**.
- c) Click **Create** to start building the project from scratch.



Step 3: Wait for Python to finish downloading and for the virtual environment to be fully set up.

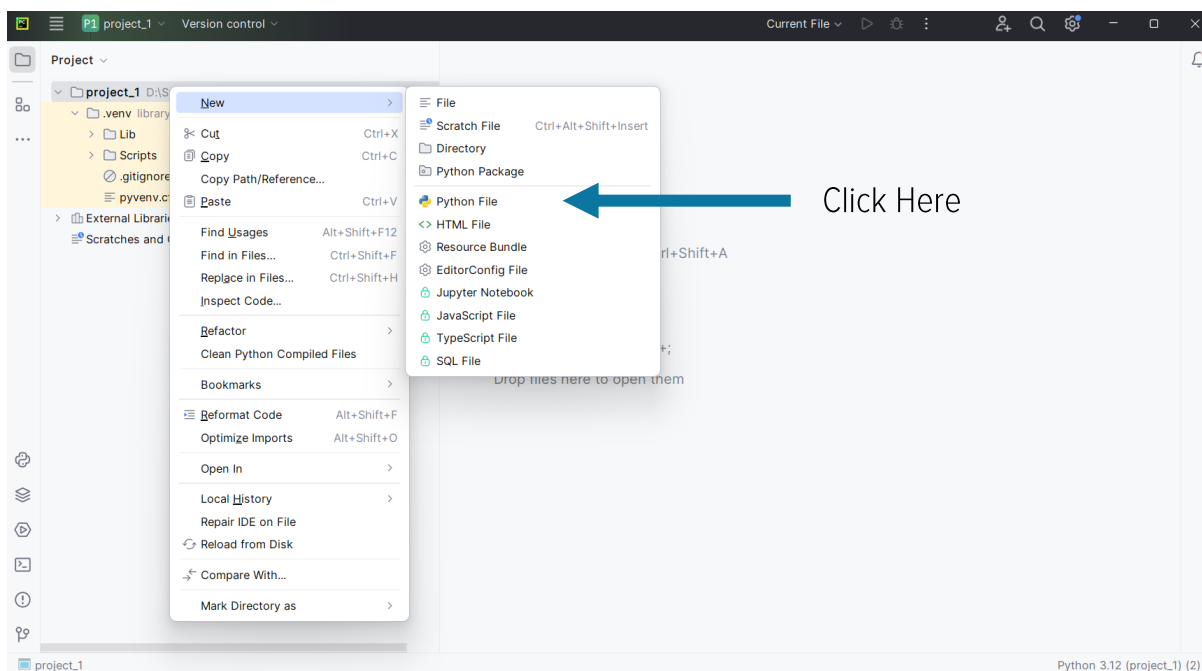
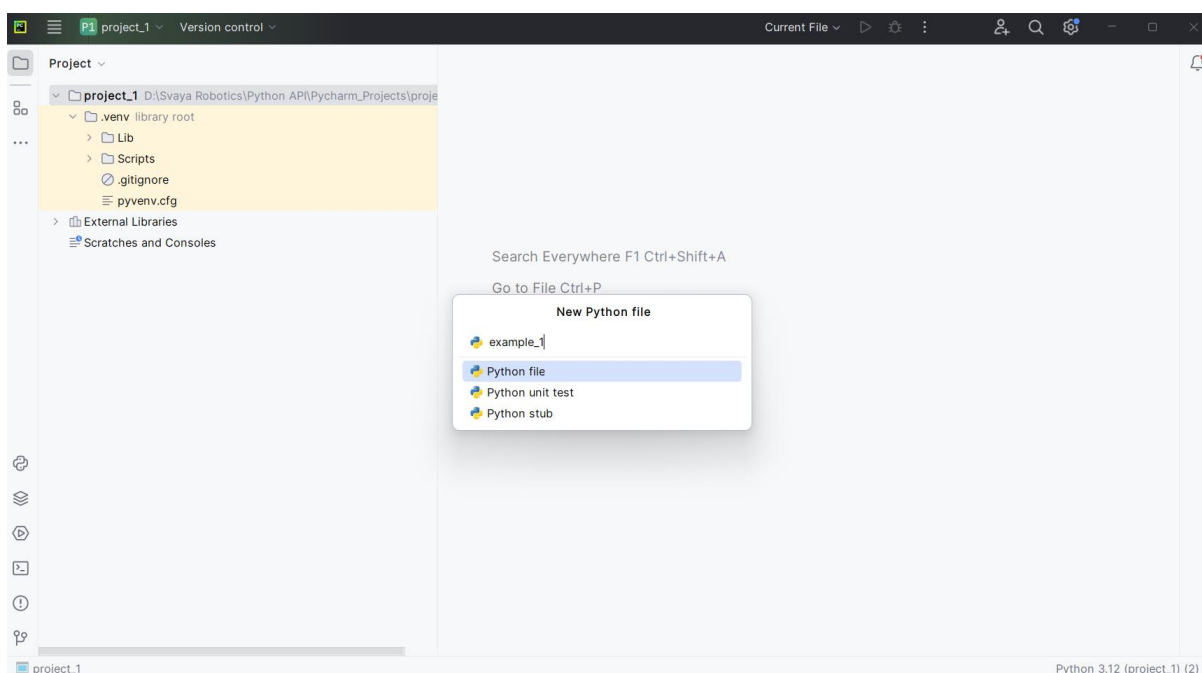


Step 4: Once the virtual environment is created successfully, the GUI of the PyCharm will look like the image shown below.

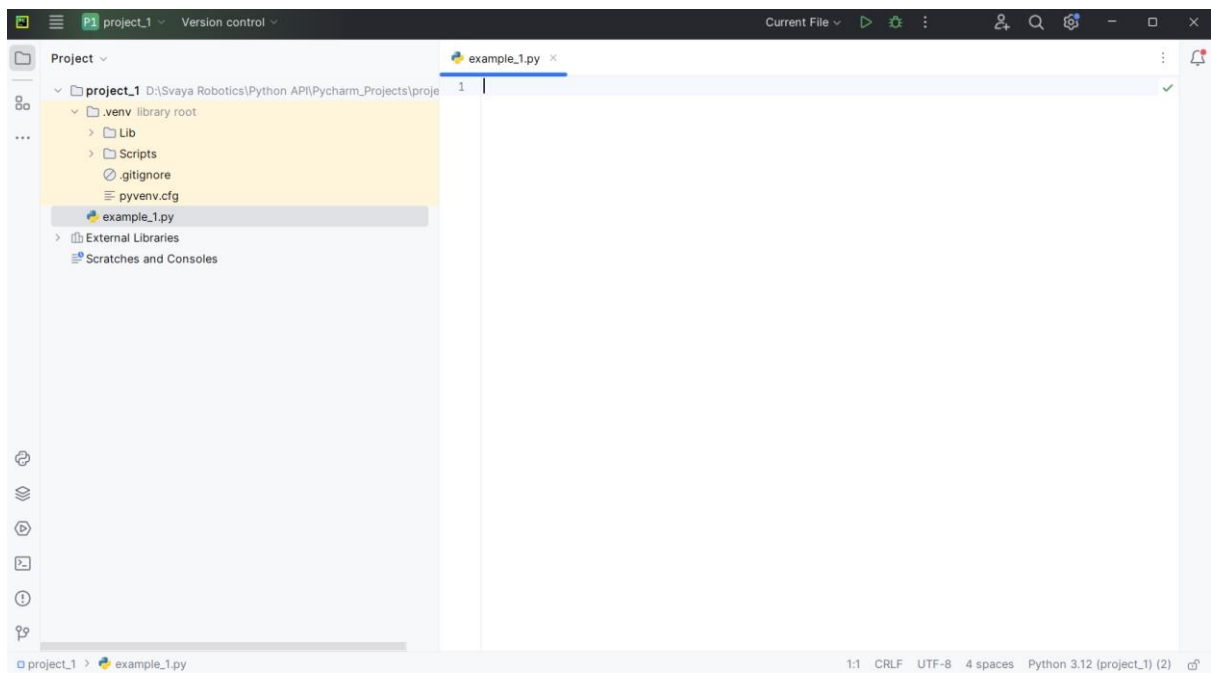


Step 5: Create a new Python file

- Right-click on the project folder, select **New**, and then click on **Python File** from the pop-up menu to create a new script file.

**Step 6:** Enter the Python file name in the dialog box and click enter.

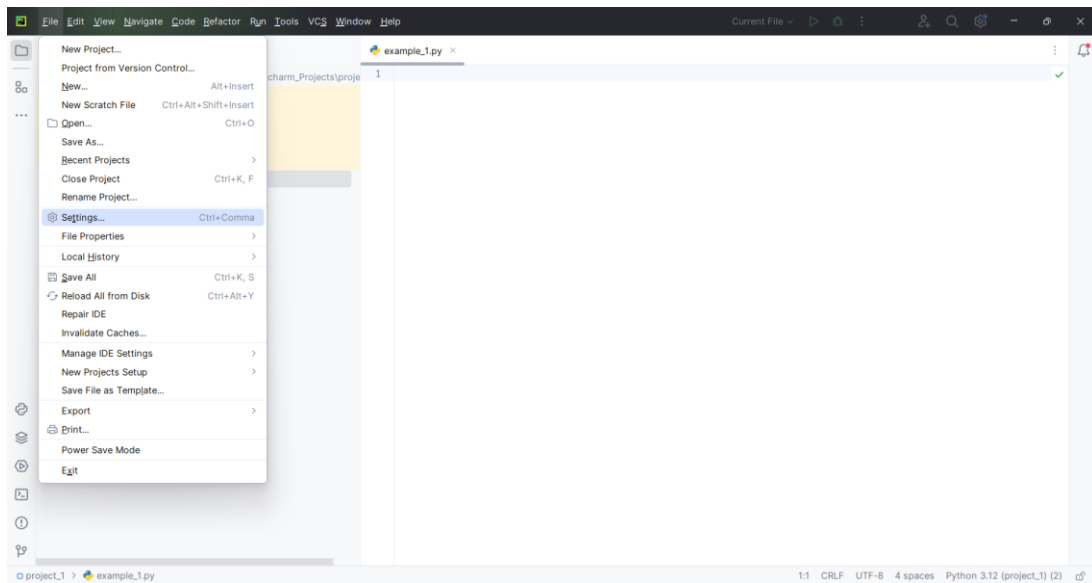
The new Python file will appear in the project tree, where the user can verify its creation.



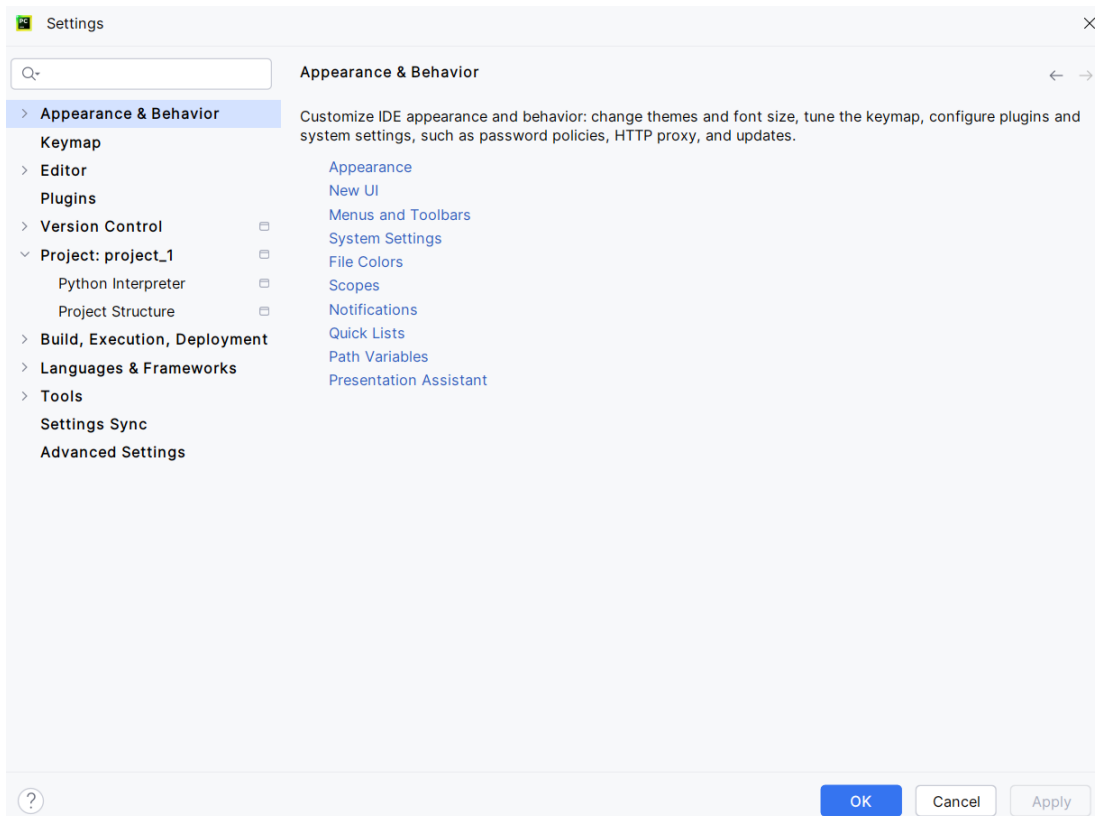
The following chapter outlines the essential libraries and packages required for the Svaya API, along with instructions for installing them.

2 Installing recommended libraries and packages

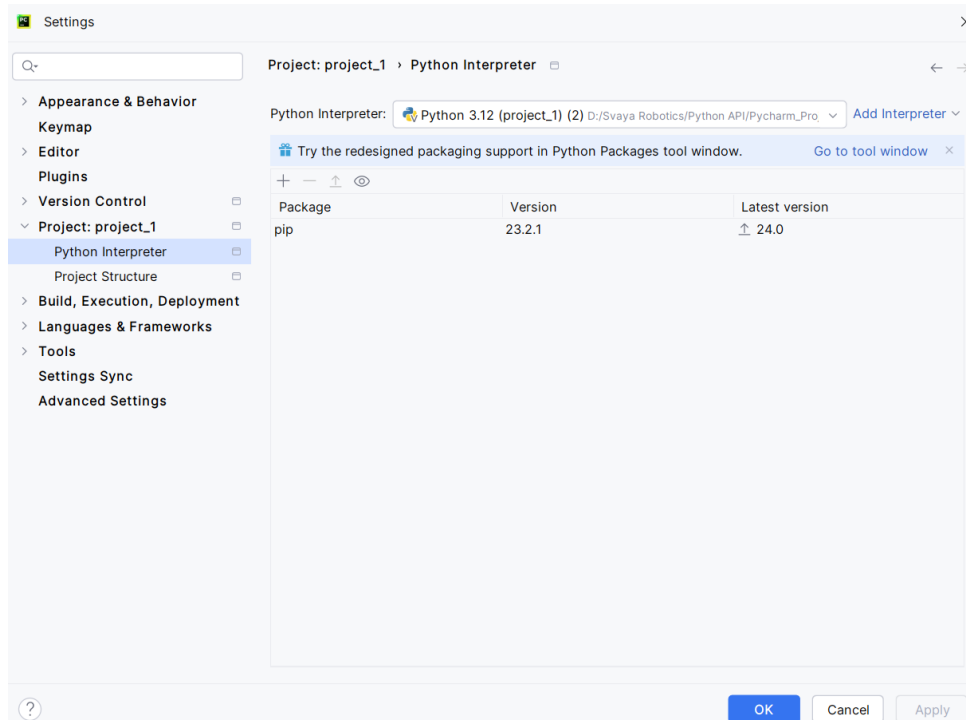
Step 1: Click the **Hamburger Menu** (File) in the top-left corner of PyCharm, then select **Settings**.



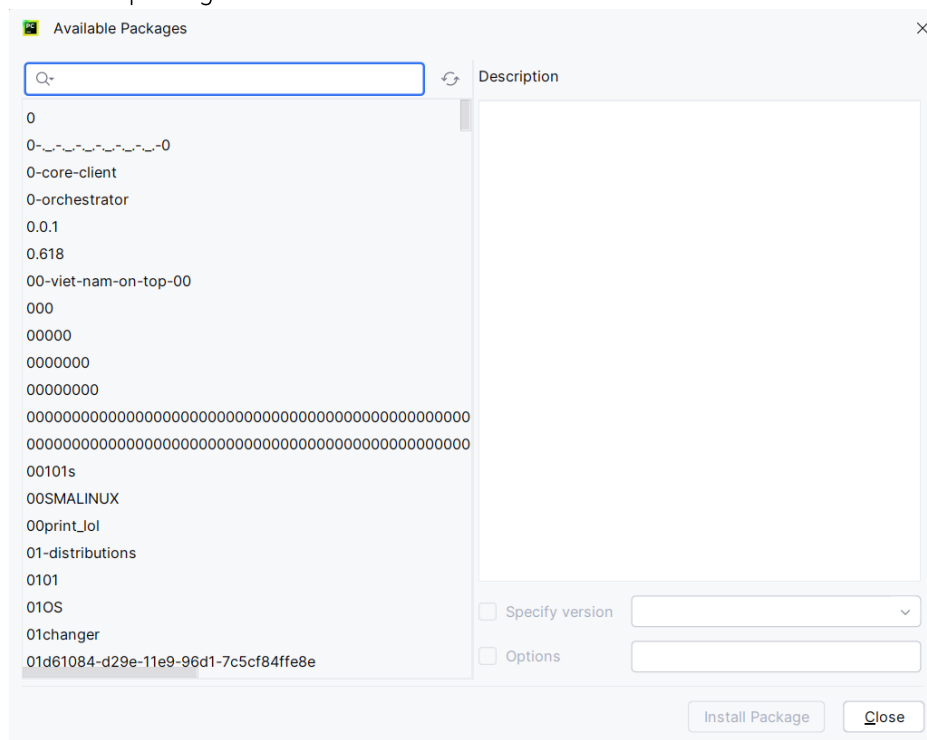
Step 2: In the **Settings** window, navigate to the **Project** section, which includes the name of the project you created earlier



Step 3: Under **Project**, click on **Python Interpreter**. Take note of the existing packages installed in the project. To install new packages, click the + icon above the pip package list.



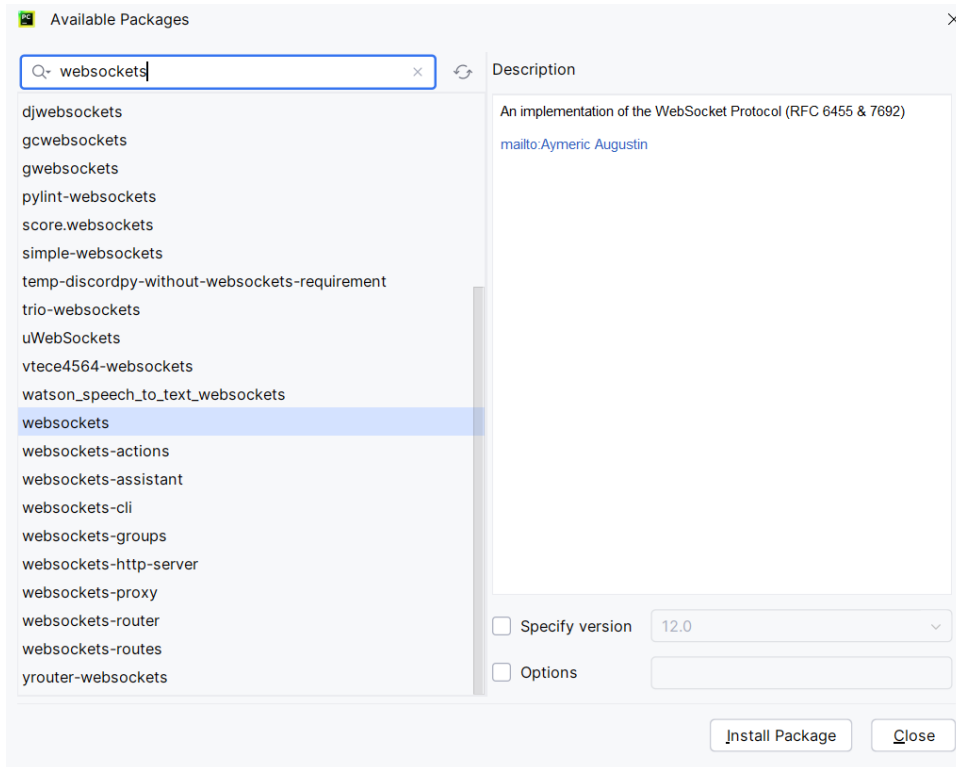
Step 4: After clicking the + icon, a dialog box titled **Available Packages** will appear, displaying a list of over 1,000 + available packages.



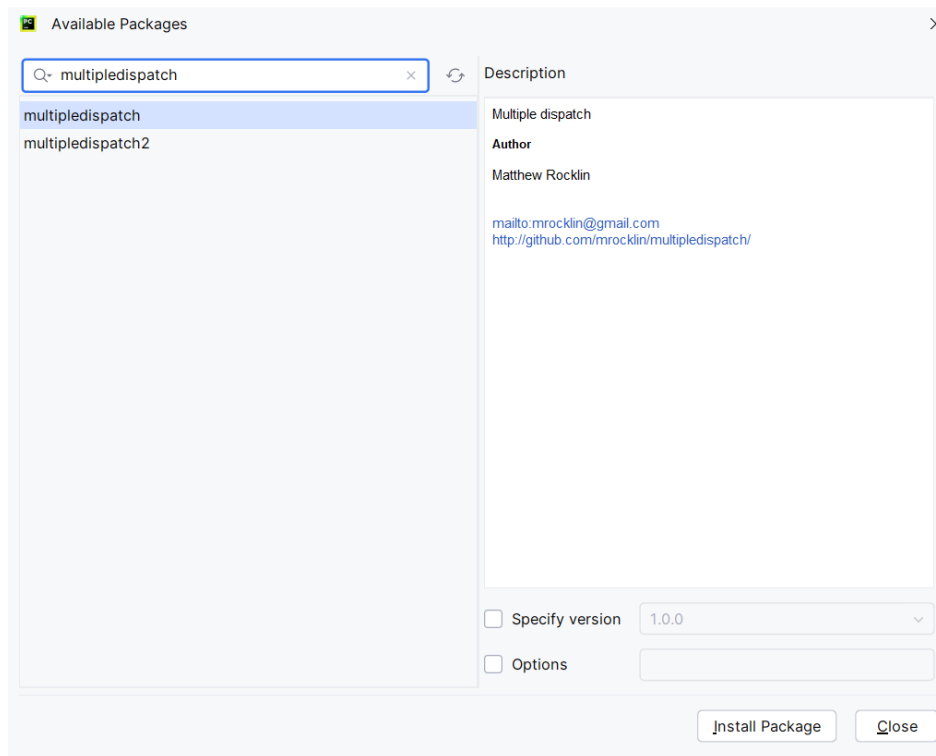
Step 5: Package Installation

- In the search bar at the top, type Websockets and confirm that the version is 12.0.
- After confirmation, click install package and wait for the installation to complete.

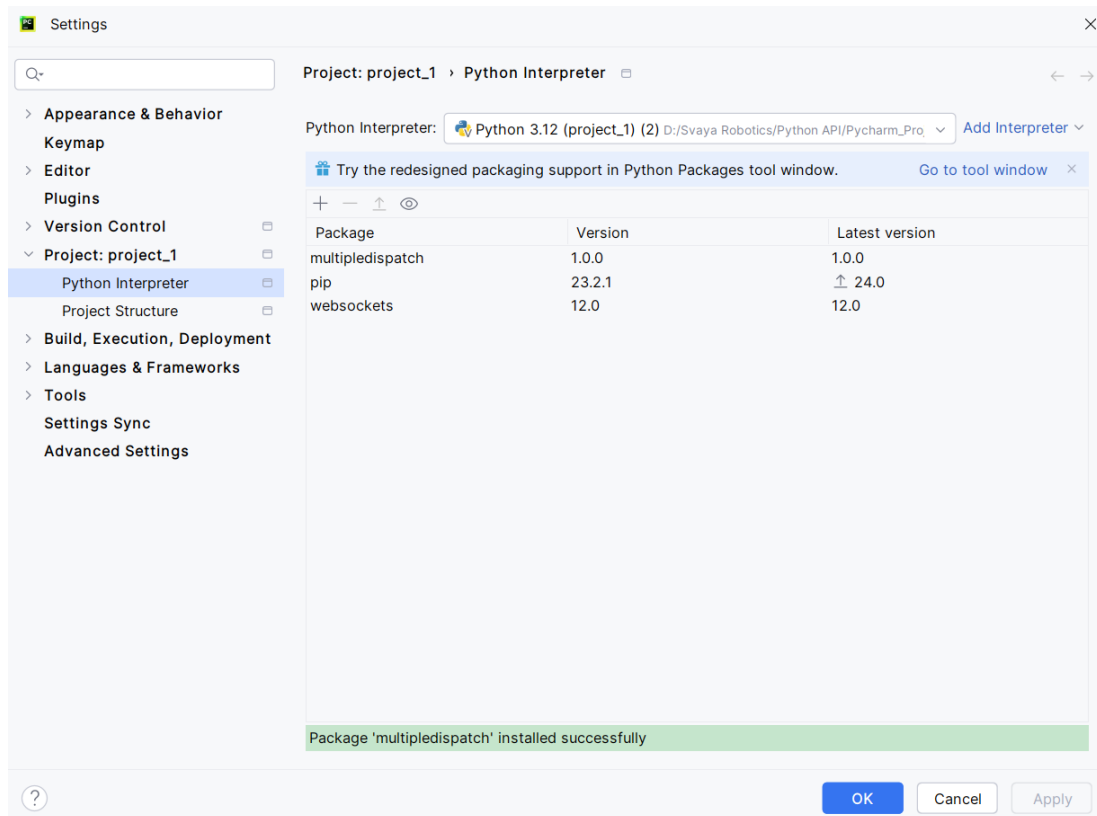
INSTALLING RECOMMENDED LIBRARIES AND PACKAGES



- c) The next package to install is `multipledispatch`, version 1.0.0.

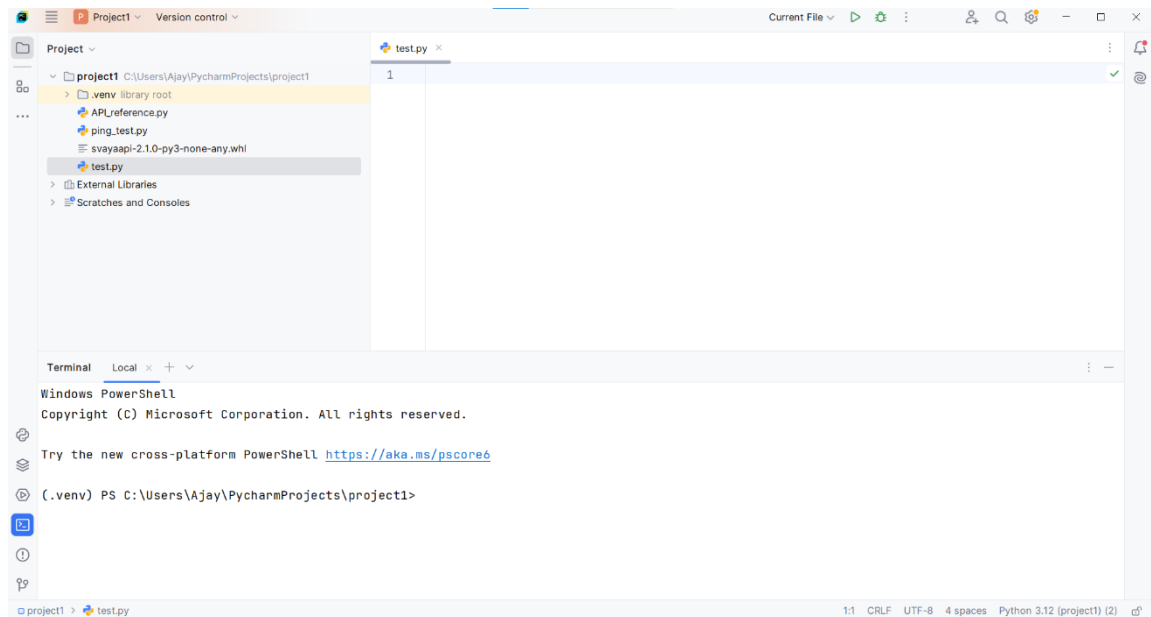


Once the installation is complete, you can check and confirm all the installed packages on the **Python Interpreter** page in the settings window.

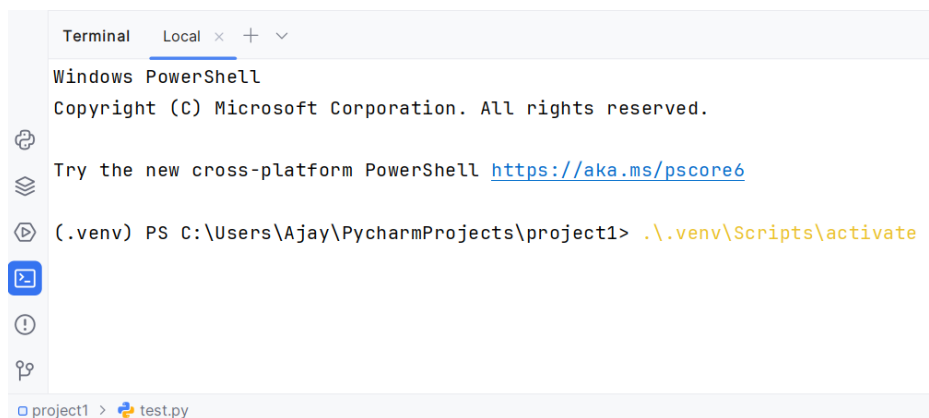


3 Svaya API Installation

Step 1: The user should open the terminal by clicking the terminal icon located at the bottom left of PyCharm.



Step 2: To activate the virtual environment, the user should type `.\venv\Scripts\activate` in the terminal. Once activated, `(.venv)` will appear at the beginning of the next line.

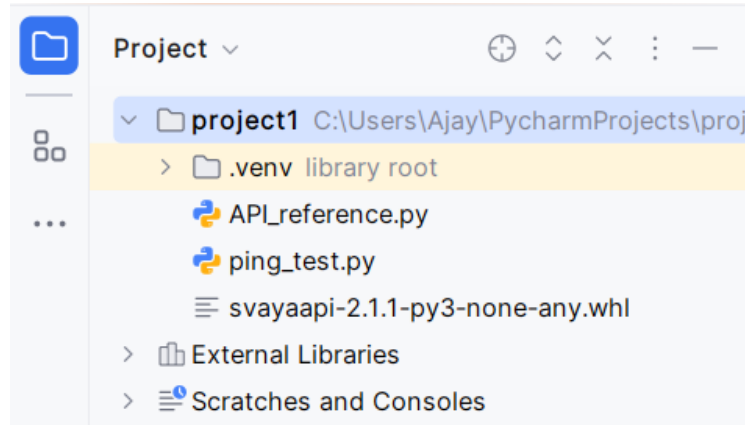


Step 3: The user should extract the contents of the provided zip file, which includes three files:

- **API_Reference** (Python file)
- **ping_test** (Python file)
- **SvayaAPI** (WHL file)

The user should then copy both Python files and the WHL file and paste them into the respective project folder.

The following screenshot shows the window after pasting the files into the project folder.



Step 4: With the virtual environment activated, the user can enter the following command in the terminal

```
Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

(.venv) PS C:\Users\Ajay\PycharmProjects\project1> .\.venv\Scripts\activate
(.venv) PS C:\Users\Ajay\PycharmProjects\project1> pip install .\svayaapi-2.1.1-py3-none-any.whl
Processing c:\users\ajay\pycharmprojects\project1\svayaapi-2.1.1-py3-none-any.whl
Installing collected packages: svayaapi
Successfully installed svayaapi-2.1.1
```

to install the WHL file: **pip install .\svayaapi-2.1.1-py3-none-any.whl**

The Svaya API and its required dependencies have now been successfully installed. With the virtual environment set up, packages installed, and necessary files in place, the project is ready for development and interaction with the Svaya robot.

4 API Commands

The following are the API commands that showcase the Svaya Robot capabilities using the API functionality.

4.1 System Initialization

These commands are used to set up the robot, establish a connection with the backend, and control the robot's operational state (enabled or disabled).

4.1.1 initialize()

Command: `SvayaApi().initialize(IP, error_callback)`

This command establishes communication between the robot and the backend server using the specified IP address.

- Param – IP: IP address of the backend
- Type – IP: String
- Param – error_callback: Function to handle asynchronous error messages and fault notifications
- Type – error_callback: Function
- Return: No return value

Example: To connect with localhost, call

```
IP = "192.168.1.10"
apiObj = SvayaApi()
apiObj.initialize(IP, error_callback)
```

 **Note:** The `initialize()` function should be called once at the beginning of the program. It is recommended to provide an `error_callback` to monitor any errors or fault states from the robot system.

Cautions:

- Always call this once at the beginning of the program.
- Do not run the UI and API simultaneously. This causes synchronization issues that may lead to control conflicts, program overrides, or motion faults.

4.1.2 enableRobot()

Command: `SvayaApi().enableRobot()`

This command activates the robot by disengaging its brakes, allowing it to perform motion operations.

- Return: No return value

 **Do:** Call only after successful initialization.

 **Caution:** It enables the robot which makes it to start virtual mode. `switchToReal()` makes robot to go real.

4.1.3 disableRobot()

Command: SvayaApi().disableRobot()

This command deactivates the robot by engaging its brakes. This ensures the robot remains stationary and safe.

- Return: No return value

 **Do:** Use this before shutdown, switching between programs or changing to Virtual mode.

4.1.4 error_callback()

This function is used to monitor and handle robot status messages and fault conditions during both initialization and runtime. It is useful for detecting and responding to events such as collision detection, E-Stop triggers, or Ethercat faults. The callback track fault priority using a priority-based system and can be customized to take action based on the error type.

- Param – error_priority: Priority level of the error. Can be "info", "medium", or "high".
- Type – error_priority: String
- Param – error_msg: The message describes the error
- Type – error_msg: String
- Param – error_status: Indicates whether the error condition is "SET" (active) or "CLEAR" (resolved).
- Type – error_status: String

Example of collision detection error handling:

```
error_flag = False
no_of_errors = 0
main_gen = None

def error_callback(error_msg, error_priority, error_status):
    global error_flag, no_of_errors
    print("Error:", error_priority, error_msg, error_status)

    if error_priority == "info":
        return

    elif error_priority == "medium":
        if error_msg == "Collision detected":
            if error_status == "SET":
                error_flag = True
                no_of_errors += 1
            elif error_status == "CLEAR":
                error_flag = False
                no_of_errors -= 1

    else: # High/Critical
        print("Critical Error! Execution should be halted.")
```

 **Do:** Use this to monitor faults states like E-Stop triggers, Ethercat faults, collision detection etc.

 **Note:** The user can customize the callback to handle additional fault types as mentioned in the **Svaya – User Manual**, using string matching on `error_msg`.

 **Warning:** If the robot enters a fault state, the callback enables program to detect and log it. Once the fault clears, the robot goes to a disabled state and can be re-enabled via **`enableRobot()`**.

 **Cautions:** Failed to assign this callback in `initialize(ip, error_callback)` will ends in undetected faults, especially during emergency scenarios like E-Stop or Ethercat faults.

4.2 Developer mode

Developer mode allows users to take direct control of the robot's torque behaviour. In this mode, users can send torque commands directly to the robot joints something that is not possible in standard operation mode.

4.2.1 enableDeveloperMode()

Command: `SvayaApi().enableDeveloperMode()`

This command activates developer mode, giving the user access to major control features, such as sending torque commands directly to the robot joints.

- Return: No return value



Note: When gravity mode is enabled, the robot's weight is internally compensated via torque offsets. The user should only provide driving/control torque.



Warning: Developer mode disables built-in motion safety systems. Sending incorrect torque values can lead to sudden, uncontrolled movements that may cause damage or injury.



Do: Use only in controlled, safe environments. Enable gravity mode before using this command.

4.2.2 disableDeveloperMode()

Command: `SvayaApi().disableDeveloperMode()`

This command turns off developer mode, restoring the robot to its standard control logic where direct torque commands are restricted.

- Return: No return value



Note: If a program ends without disabling developer mode, the robot will automatically exit developer mode after approximately 100 milliseconds.



Caution: If developer mode remains active while torque commands are no longer sent, the robot enters gravity compensation state.



Do: Always call this command after completing torque-control tasks to restore normal operation.

4.2.3 getDeveloperModeStatus()

Command: `SvayaApi().getDeveloperModeStatus()`

This command returns the status of the developer mode, whether it is enabled or disabled.

- Return: Boolean – True: If developer mode is active, False: If developer mode is not active



Note: Use this check before sending torque commands to avoid API errors.

4.2.4 sendTorqueCommand()

Command: SvayaApi().sendTorqueCommand(torque_values)

This command sends user defined torque values directly to the robot joints. It can only be used when developer mode is enabled.

- Param – torque_values: List of 6 values of torque for each joint
- Type – torque_values: List[float]

Rated joint torque:

- Joint 1 & 2: ± 183 Nm
- Joint 3: ± 107 Nm
- Joint 4, 5 & 6: ± 59 Nm

Warnings:

- Sending incorrect torque values may cause unpredictable robot behaviour or hardware damage. Always test in a safe environment.
- If the commanded torque is too high, the joint may exceed its velocity limits, causing the robot to enter a fault state.

Do: Test with low torque values initially.

Don'ts:

- Don't exceed joint-specific torque limits. Exceeding these limits causes faults and may damage actuators.
- Don't leave the robot in developer mode unattended. Always supervise the robot when torque control is active.
- Don't restart the program without confirming that robot gravity mode is inactive.

Procedure sending reference torque to 6-axis Robot:

Follow these steps to safely send reference torque commands using developer mode:

1. Start the Robot.
2. Switch the robot in sequence: Disable > Enable > Manual
3. Turn ON the gravity mode by pressing the green button on Tool I/O.
4. Enable developer mode using **enableDeveloperMode()** command.
5. Start sending torque command using **sendTorqueCommand()** command by sending a list of torque values in float.

Note: The robot will automatically engage brakes in developer mode under the safety conditions:

- If joint position is exceeding defined joint limits, robot stops 10 degrees before the configured limit in the UI.
- If the joint velocity exceeds 75 percent of the configured limits in the UI.
- If the commanded torque exceeds 50 percent of the **joint rated torque**.

4.2.5 getGravitytorque()

Command: SvayaApi().getGravitytorque()

This command returns the gravity compensation torque values for each joint of the robot for this current joint space measured on the motor side. The returned values are without inertial torque (No velocity and acceleration component).

- Return: List of gravity compensation torque.
- Rtype: List [float]

Sample program for developer mode:

```
import time
from SvayaAPI.CartesianPose import CartesianPose
from SvayaAPI.Frame import Frame
from SvayaAPI.clientApi import SvayaApi
from SvayaAPI.Trajectory import Trajectory
import tkinter as tk
import queue

ip = "192.168.20.203"

def error_callback(error_priority, error_msg, error_status):
    print("Error:", error_priority, error_msg, error_status)
    if error_priority == "info":
        print("Info:", error_msg, error_status)
    elif error_priority == "medium":
        print("Medium:", error_msg, error_status)
    elif error_priority == "high":
        print("High:", error_msg, error_status)

cobot = SvayaApi()
cobot.initialize(ip, error_callback)
print("The robot is initialized successfully")

cobot.disableRobot()
cobot.waitForTime(0.5)
cobot.enableRobot()
cobot.waitForTime(0.5)
cobot.switchToReal()
print("Robot is in real mode")
cobot.waitForTime(0.5)

# Developer mode operations
print("Developer mode status:", cobot.getDeveloperModeStatus())

state = True
while state:
    print("\nMenu:")
    print("1 - Send torque command for 3 seconds")
    print("2 - Disable developer mode")
    print("3 - Exit program")
    user_input = input("Select an option: ")
```

```

if user_input == "1":
    while(not cobot.getDeveloperModeStatus()):
        cobot.enableDeveloperMode()
        cobot.waitForTime(0.05)
    if cobot.getDeveloperModeStatus() == True:
        print("Sending torque command for 10 seconds")
        start_time = time.time()
        while time.time() - start_time < 10:
            cobot.sendTorqueCommand([0.0, 0.0, 0.0, 0.0, 0.0, 0.0])
            print("Gravity compensation torque:", cobot.getGravitytorque())
            cobot.waitForTime(0.05)
        print("Torque command stopped.")
    else:
        print("Developer mode is not enabled.")

elif user_input == "2":
    cobot.disableDeveloperMode()
    print("Developer mode status:", cobot.getDeveloperModeStatus())

elif user_input == "3":
    print("Exiting program...")
    state = False

else:
    print("Invalid option.")

cobot.__del__()
print("Robot connection closed.")

```

4.3 Mode Switching

These commands are used for switching between various robot operation modes such as real, virtual, and automatic. Mode switching defines how the robot behaves whether it's simulating virtually or executing them physically.

4.3.1 switchToReal()

Command: `SvayaApi().switchToReal()`

This command switches the robot from virtual to real-world operation mode, enabling physical motion. Which is used during manual teaching and while task execution.

- Return: No return value



Do: Call **switchToReal()** only when robot is idle, and the work cell is secure. User can enable gravity only when robot is manual state. So, it works fine until the **switchToAuto()** is called.

4.3.2 switchToVirtual()

Command: `SvayaApi().switchToVirtual()`

This command switches the robot into virtual mode, where all motion commands are simulated without actual movement.

- Return: No return value



Caution: Virtual mode does not physically move the robot. It simulates motion and only reflects changes on the Svaya UI.



Do: Use **switchToVirtual()** for testing and simulation before switching to real hardware execution.

4.3.3 switchToAuto()

Command: `SvayaApi().switchToAuto()`

This command switches the robot into automatic mode, where it can execute pre-programmed tasks.

- Return: No return value



Do: Call **switchToAuto()** only after ensuring the robot is in real mode and ready for automatic execution.



Don't: Don't hold the robot in gravity mode and switch to auto without following proper sequence.

4.4 Feedback and Sensing

These commands allow the robot to collect and process various types of sensor and system data. This includes force/torque sensing, which measures the external forces and moments acting on the robot; motor torque, which provides the torque values generated by each motor; and joint feedback, which includes information like joint positions, velocities.

4.4.1 getJointValues()

Command: `SvayaApi().getJointValues()`

This command returns the current joint angles of the robot. Each value in the list corresponds to a specific joint, typically measured in degrees.

- Return: List of joint angles
- R-Type: List[float]

4.4.2 getJointVelocities()

Command: `SvayaApi().getJointVelocities()`

This command returns the current velocity of the robot's joints. The values are given in a list, where each element represents the velocity of a specific joint, usually in degrees/second.

- Return: List of joints velocity
- R-Type: List[float]

4.4.3 getCartPose()

Command: `SvayaApi().getCartPose()`

This command returns an object that contains the current cartesian position and orientation of the robot's joints. The position is typically represented by x, y, and z coordinates, while the orientation is given as roll, pitch, and yaw values.

- Return: Object of cartesian pose class
- R-Type: Cartesian pose (custom class with fields: x, y, z, roll, pitch, yaw)

Example:

```
apiObj = SvayaApi()
cart_pose = apiObj.getCartPose() # To Get the Cartesian Values.
print("X:", cart_pose.x, "mm")
print("Y:", cart_pose.y, "mm")
print("Z:", cart_pose.z, "mm")
print("RX:", cart_pose.roll, "degrees")
print("RY:", cart_pose.pitch, "degrees")
print("RZ:", cart_pose.yaw, "degrees")
```

4.4.4 getCartPoseArray()

Command: `SvayaApi().getCartPoseArray()`

This command returns the current position and orientation of the robot's joint in cartesian coordinates. The result is given as an array where the first three values represent the position (x, y, and z), and the next three values represent the orientation as roll, pitch, and yaw.

- Return: List with x, y, z, roll, pitch and yaw
- R-Type: List[float]

4.4.5 getQuaternion()

Command: `SvayaApi().getQuaternion()`

This command returns the robot's current orientation in quaternion format, which provides a more stable 3D rotational representation.

- Return: Object of quaternion class
- R-Type: Quaternion pose (custom class with fields: x, y, z, quat_x, quat_y, quat_z, quat_w)

Example:

```
apiObj = SvayaApi()
quaternion = apiObj.getQuaternion()
print("X:", quaternion.x, "mm")
print("Y:", quaternion.y, "mm")
print("Z:", quaternion.z, "mm")
print("QuatX:", quaternion.quat_x, "radians")
print("Quat_Y:", quaternion.quat_y, "radians")
print("Quat_Z:", quaternion.quat_z, "radians")
print("Quat_W:", quaternion.quat_w, "radians")
```

4.4.6 getQuaternionArray()

Command: `SvayaApi().getQuaternionArray()`

This command returns the robot's position and orientation as a quaternion in an array. The first three values represent the position: x, y, and z. The next four values represent the orientation as a quaternion: quat_x, quat_y, quat_z, and quat_w. This format allows for accurate 3D positioning and rotation representation.

- Return: List with x, y, z, quat_x, quat_y, quat_z and quat_w
- R-Type: List [float]

4.4.7 getFTTorqueData()

Command: `SvayaApi().getFTTorqueData()`

This command returns the current torque values from the drives along the X, Y, and Z axes, represented as Tx, Ty, and Tz. These values indicate the torque acting on the robotic arm in the respective directions: X, Y, and Z. This information helps in monitoring and adjusting the arm's movements and force distribution.

- Return: List of torque of drives

- R-Type: List [float]

4.4.8 getFTForceData()

Command: SvayaApi().getFTForceData()

This command returns the current force values exerted by the drives along the X, Y, and Z axes, represented as Fx, Fy, and Fz. These values correspond to the forces acting on the robotic arm in the X, Y, and Z directions.

- Return: List of force of drives
- R-Type: List [float]

⚠ Warning: getFTTorqueData() and getFTForceData() commands only works if an FTS is installed.

✗ Don't: Use FTS torque values for safety-critical applications without verification and calibration.

4.4.9 getMotorTorqueData()

Command: SvayaApi().getMotorTorqueData()

This command returns the torque data from robot motors.

- Return: List of torque of motors
- R-Type: List [float]

4.4.10 getJTSBiasedData()

Command: SvayaApi().getJTSBiasedData()

This command returns the torque values for each joint of the robot, indicating the rotational forces acting at the joints during movement.

- Return: List of torque of joints
- R-Type: List [float]

4.5 Delays and Synchronization

These commands manage time delays and synchronization during robot operation, ensuring smooth coordination between tasks. They allow the robot to pause for a specified duration or synchronize through inputs ensuring precise control over movements.

4.5.1 waitForInput()

Command: `SvayaApi().waitForInput(id,value)`

This command pauses the robot's execution until a specific digital input reaches the desired state (True or False). It is commonly used to synchronize the robot with external inputs like sensors, switches, or other automation components.

- Param - id: Number of the digital input
- Type - id: Int, range [1, 16]
- Param - value: State of the digital output, either True or False
- Type - value: Boolean

Example: To wait until digital input 1 to turn True,

```
apiObj = SvayaApi()  
apiObj.waitForInput(1, True)
```

4.5.2 waitForTime()

Command: `SvayaApi().waitForTime(value)`

This command pauses robot execution for a specified amount of time, measured in seconds.

- Param - value: Duration for which the execution wants to wait
- Type - value: Int or float

Example: To wait for 5 seconds,

```
apiObj = SvayaApi()  
apiObj.waitForTime(5.0)
```

4.6 Configuration and control

Configuration and control functions are used to set up and manage the robot's operational settings. These include configuring output signals, adjusting system parameters. These functions ensure that the robot operates according to the desired setup.

4.6.1 setDigitalOutput()

Command: `SvayaApi().setDigitalOutput(pinId, mode, value)`

This command sets the value of digital output to True/False.

- Param - id: Number of the digital output
- Type - id: Int, range [1, 16] (both 1 and 16 included)
- Param - mode: Mode of the digital output (PNP or NPN)
- Type - mode: Boolean (False for PNP, True for NPN)
- Param - value: The state of the digital output (True for ON, False for OFF)
- Type value: Boolean (True or False)

Example: To set digital output 1 as True and digital output 2 as False,

```
apiObj = SvayaApi()
apiObj.setDigitalOutput(1, False, True)
apiObj.setDigitalOutput(2, False, False)
```

4.6.2 setAnalogOutput()

Command: `SvayaApi().setAnalogOutput(pinId, mode, value)`

This command sets the analog output value.

- Param - pinId: Number of the analog output
- Type - pinId: Int, range [1, 4] (both 1 and 4 included)
- Param – modeType: Type of the analog output (Volts or Amperes)
- Type - mode: Int (0 for Volts, 1 for current)
- Param - value: The value of the analog output
- Type - value: Float
 - If mode is 0 (Volts), the value range is [0, 10] V
 - If mode is 1 (current), the value range is [4.0, 20.0] mA

Example: To set analog output 1 to 5 volts,

```
apiObj = SvayaApi()
apiObj.setAnalogOutput(1, 0, 5.0)
```

4.6.3 setToolDigitalOutput()

Command: `SvayaApi().setToolDigitalOutput(pinId, mode, value)`

This command sets the tool digital output value to True/False.

- Param - pinId: Number of the Tool's digital output

- Type - pinId: Int (1 or 2)
- Param - mode: Mode of the digital output (PNP or NPN)
- Type - mode: Boolean (False for PNP, True for NPN)
- Param - value: The state of the digital output (True for ON, False for OFF)
- Type - value: Boolean (True or False)

Example: To open actuate the tool at two different positions,

```
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 0.0, 90.0, 0.0, 0.0, 150.0])
apiObj.setToolDigitalOutput(1, False, True)
apiObj.waitForTime(0.5)
apiObj.setToolDigitalOutput(2, False, False)
apiObj.moveJoints([90.0, 0.0, 90.0, 0.0, 0.0, -150.0])
apiObj.setToolDigitalOutput(2, False, True)
apiObj.waitForTime(0.5)
apiObj.setToolDigitalOutput(1, False, False)
```

4.6.4 setJogSpeed()

Command: `SvayaApi().setJogSpeed(speed)`

This command sets speed of the robot in jogging. It takes integer which is percentage of max robot speed (max speed is 90 degrees per second), user wants to set.

- Param - speed: Speed of the robot
- Type - speed: Takes integer or float, range [1,100]

Example: To start robot jogging at 50 percentage of max speed,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(50)
```



Do: Always set a safe jog speed before using jog commands.

4.6.5 setJointJogStep()

Command: `SvayaApi().setJointJogStep(step)`

This command sets the step value to the user-given value which is used in step jogging.

- Param - step: No of degrees joint must jog in one step
- Type - step: Takes float value, range [1,10]

Example: To set step to 1 degree,

```
apiObj = SvayaApi()
apiObj.setJointJogStep(1.0)
```

4.6.6 setCartesianJogStep()

Command: `SvayaApi().setCartesianJogStep(step)`

This command sets the step value to the user given value which is used in cartesian step jogging.

- Param - step: No of mm(millimetres) must jog in one step
- Type - step: Takes float value, range [1,10]

Example: To set step to 5 mm,

```
apiObj = SvayaApi()
apiObj.setCartesianJogStep(5.0)
```

4.6.7 setFTBiasState()

Command: `SvayaApi().setFTBiasState(bias,sampling_command)`

This command sets the bias state for force/torquers.

- Param - bias: Axia bias or not
- Type - bias: Boolean True for bias and False for not bias
- Param - sampling command: Frequency of data (500hz, 1000hz, 2000hz, 3000hz)
- Type - sampling command: Int (0-500hz, 1-1000hz, 2-2000hz, 3-3000hz)

Example:

```
apiObj = SvayaApi()
apiObj.setFTBiasState(True, 3)
time.sleep(1)
apiObj.setFTBiasState(False, 3)
```

To ensure accurate force/torque measurements, initiate biasing when the robot is stationary to zero out existing offsets or loads. After a short delay, disable biasing to lock in the reference and begin normal sensor operation. This process stabilizes sensor readings and improves measurement reliability.

 **Warning:** Only works if Force/Torque Sensor (FTS) is installed.

 **Do:** Bias only when robot is still and unloaded.

 **Don't:** Use this command while robot is in motion or under load.

4.6.8 setPayload()

Command: `SvayaApi().setPayload(payload_mass, payload_com)`

This command sets the payload of the robot.

- Param - payload_mass: Mass of the payload in kilograms
- Type - payload_mass: float
- Param - payload_com: Centre of mass of the payload as a list of three floats [x, y, z] in millimetres
- Type - payload_com: list[float]

Example: To set a payload with mass 5.0 kg, centre of mass at [0.1, 0.0, 0.2] millimetres,

```
apiObj = SvayaApi()
apiObj.setPayload(5.0, [0.1, 0.0, 0.2])
```

 **Warning:** Incorrect payload setting can affect motion smoothness and safety systems.

 **Do:** Measure payload and centre of mass accurately.

4.6.9 setRobotBaseFrame()

Command: `SvayaApi().setRobotBaseFrame(Frame)`

This command sets the robot's base frame relative to the world coordinate system, where the origin is [0, 0, 0]. It is used when the robot is mounted on a different position or orientation than the world origin, allowing to define the robot's actual base position in space.

- Param – Frame: The frame representing the robot's mounting with respect to the world frame
- Type - Frame: Base frame object

 **Do:** Use this command when the robot is installed on a platform or surface that is offset or rotated from the default world origin.

Example:

```
apiObj = SvayaApi()
base_frame = Frame(-50.0, -50.0, 0.0, 0.0, 0.0, 0.0)
apiObj.setRobotBaseFrame(base_frame)
```

4.6.10 resetRobotBaseFrame()

Command: `SvayaApi().resetRobotBaseFrame()`

This command resets the robot's base frame to the default world coordinate system, where the origin is set to [0, 0, 0]. This is useful when the base frame is previously modified (e.g., using `setRobotBaseFrame()`), and now want to return the robot's reference frame back to its original setting.

- Return: No return value

 **Do:** Use this to change by the robot base frame.

4.6.11 setJogUserFrame()

Command: `SvayaApi().setJogUserFrame(user_frame)`

This command sets a custom user frame for cartesian operations. A user frame defines a local coordinate system that makes it easier to move or control the robot relative to a specific workspace, rather than the default robot base frame. It is especially useful in applications where the robot needs to interact with tools, fixtures, or workpieces that are offset from the robot's base.

- Param – user_frame: The coordinate frame with respect to robot's base frame/world frame
- Type – user_frame: Frame object

 **Do:** Use when working on specific fixtures or areas to simplify programming as the jog directions (X, Y, Z) will now be interpreted relative to the new frame at workspace instead of robot's base frame.

Example:

```
apiObj = SvayaApi()
user_frame = Frame(250.0, -50.0, 100.0, 0.0, 0.0, 0.0)
apiObj.setJogUserFrame(user_frame)
```

4.6.12 resetJogUserFrame()

Command: `SvayaApi().resetJogUserFrame()`

This command resets the active user frame back to the default robot base frame. It is used when a custom user frame was previously set using `setJogUserFrame()`.

- Return: No return value

4.6.13 setJogToolFrame()

Command: `SvayaApi().setJogToolFrame(tool_frame)`

This command sets the tool frame used during cartesian operations. A tool frame defines the position and orientation of the tool with respect to the robot's flange. By setting a custom tool frame, jog motions can be done relative to the tool's axes, which is useful for applications like welding, painting, or assembly.

- Param – tool_frame: The frame of tool tip with respect to tool flange
- Type – tool_frame: Frame object

 **Do:** Upon creating the tool frame, the jogging and cartesian position changes based on the geometry of the tool frame that got updated.

Example:

```
apiObj = SvayaApi()
tool_frame = Frame(100.0, 0.0, 30.0, 0.0, 0.0, 0.0)
apiObj.setJogUserFrame(tool_frame)
```

4.6.14 resetJogToolFrame()

Command: `SvayaApi().resetJogToolFrame()`

This command resets the jogging tool frame back to its default state, which aligns with the robot's flange coordinate system. Once reset, all jogging movements will be relative to the robot flange.

- Return: No return value

4.6.15 setJogUserAndToolFrame()

Command: `SvayaApi().setJogUserAndToolFrame(user_frame, tool_frame)`

This command sets both the user frame and tool frame simultaneously for cartesian motion. It helps simplify setup when both frames need to be updated together, ensuring all jog commands are performed correctly relative to the defined coordinate systems.

- Param – user_frame: The user frame with respect to robot base frame/world frame
- Type – user_frame: Frame object
- Param – tool_frame: The tool frame with respect to tool flange

- Type - tool_frame: Frame object

Example:

```
apiObj = SvayaApi()
user_frame = Frame(250.0, 50.0, 20.0, 0.0, 0.0, 0.0)
tool_frame = Frame(100.0, 0.0, 30.0, 0.0, 0.0, 0.0)
apiObj.setJogUserFrame(user_frame,tool_frame)
```

4.6.16 resetJogUserAndToolFrame()

Command: `SvayaApi().resetJogUserAndToolFrame(user_frame, tool_frame)`

This command resets both the user frame and tool frame to their default values. The robot will revert to using the world frame as the user coordinate system and the flange as the default tool reference.

- Return: No return value

4.7 Motion Control

These commands manage the robot's movement and trajectory. They enable the robot to perform precise, smooth, and coordinated motions during tasks such as pick-and-place, assembly, and inspection. Motion control is fundamental for navigation, safety, and synchronization with external processes.

4.7.1 goToHome()

Command: `SvayaApi().goToHome()`

This command moves the robot to its predefined home position, where all joint angles are set to zero (0). The movement is carried out at jog speed, which is a percentage of the robot's global speed commonly set to 90°/s by default.

Example: To move robot to home position,

```
apiObj = SvayaApi()
api.goToHome()
```

4.7.2 goToHome()

Command: `SvayaApi().goToHome(time)`

This command moves the robot to its home position within a specified duration. The time parameter defines how long the robot should take to reach the home position. This is useful for timed operations.

- Param - time: Time duration to reach the home position
- Type - time: Float, must be greater than 0

Example: To move robot to home position in 10 seconds,

```
apiObj = SvayaApi()
api.goToHome(10.0)
```

4.7.3 goToHome()

Command: `SvayaApi().goToHome(velocity, acceleration)`

This command moves the robot to its home position with the specified velocity and acceleration values. These parameters allow for precise control over the speed and smoothness of the movement.

- Param - velocity: Velocity in degrees per second
- Type - velocity: Float, range [1, 180]
- Param - acceleration: Acceleration in degrees per second squared
- Type - acceleration: Float, range [1, 180]

Example: To move robot to home position with velocity 60°/s and 90°/s²,

```
apiObj = SvayaApi()
api.goToHome(60.0, 90.0)
```


4.7.4 goToHomeAsync()

Command: `SvayaApi().goToHomeAsync()`

This command moves the robot to the home position. The key feature of this command is that it runs asynchronously, meaning it starts the movement but does not wait for it to finish before continuing with other commands.

Example: To move robot to home position,

```
apiObj = SvayaApi()
api.goToHomeAsync()
```

4.7.5 goToHomeAsync()

Command: `SvayaApi().goToHomeAsync(time)`

This command moves the robot to the home position asynchronously within a specified time duration.

- Param - time: Time duration to reach the home position
- Type - time: Float, must be greater than 0

Example: To move robot to home position in 10 seconds,

```
apiObj = SvayaApi()
api.goToHomeAsync(10.0)
```

4.7.6 goToHomeAsync()

Command: `SvayaApi().goToHomeAsync(velocity, acceleration)`

This command moves the robot to the home position asynchronously, using the specified velocity and acceleration.

- Param - velocity: Velocity in degrees per second
- Type - velocity: Float, range [1, 180]
- Param - acceleration: Acceleration in degrees per second squared
- Type - acceleration: Float, range [1, 180]

Example: To move robot to home position with velocity 60°/s and 90°/s²,

```
apiObj = SvayaApi()
api.goToHomeAsync(60.0, 90.0)
```

4.7.7 moveJoint()

Command: `SvayaApi().moveJoint(jointID, jointValue)`

This command moves a specific joint of the robot by a defined angle. It allows for precise individual joint control, useful for fine-tuning positions or manual adjustments.

- Param - jointID: The joint number that must be moved
- Type - jointID: Int, range [1, 6] (both 1 and 6 included)
- Param - jointValue: Number of degrees the joint must rotate
- Type - jointValue: Float

Example: To move joint 1 by 60.00,

```
apiObj = SvayaApi()
apiObj.moveJoint(1, 60.0)
```

4.7.8 moveJointAsync()

Command: `SvayaApi().moveJointAsync(jointID, jointValue)`

This command moves the specified joint by the given angle in degrees and runs asynchronously, allowing other tasks to continue while the joint moves.

- Param - jointID: The joint number that must be moved
- Type - jointID: Int, range [1, 6] (both 1 and 6 included)
- Param - jointValue: Number of degrees the joint must rotate
- Type - jointValue: Float

Example: To move joint 1 by 60.00,

```
apiObj = SvayaApi()
apiObj.moveJointAsync(1, 60.0)
```

4.7.9 moveJoints()

Command: `SvayaApi().moveJoints(jointAngles)`

This command moves multiple joints simultaneously to the specified target angles. Each joint is adjusted to its corresponding target position in degrees, enabling coordinated control over the robot's posture.

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6

Example: To move joint 2 by 60° and joint 3 by 45°,

```
apiObj = SvayaApi()
arr = [0.0, 60.0, 45.0, 0.0, 0.0, 0.0]
apiObj.moveJoints(arr)
Or
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 60.0, 45.0, 0.0, 0.0, 0.0])
```

4.7.10 moveJoints()

Command: `SvayaApi().moveJoints(jointAngles, jointVel, jointAcc)`

This command moves the joints to the specified degrees provided by the user, with the given joint velocity (jointVel) and joint acceleration (jointAcc).

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6
- Param - jointVel: Number of degrees per second the joints must move
- Type - jointVel: Float, specifies degrees per second
- Param - jointAcc: Number of degrees per second² the joints must accelerate
- Type - jointAcc: Float, specifies degrees per second²

Example: To move joint 2 by 60° and joint 3 by 45° with joint velocity of 90°/s and joint acceleration of 100°/s²,

```
apiObj = SvayaApi()
arr = [0.0, 60.0, 45.0, 0.0, 0.0, 0.0]
jointVel = 90.0
jointAcc = 100.0
apiObj.moveJoints(arr, jointVel, jointAcc)
Or
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 60.0, 45.0, 0.0, 00.0, 0.0], 90.0, 100.0)
```

4.7.11 moveJoints()

Command: `SvayaApi().moveJoints(jointAngles, timeToReach)`

This command moves the joints to the specified degrees provided by the user and ensures the target position is reached within the specified time (timeToReach).

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6
- Param - timeToReach: Number of seconds the joints should take to reach their joint values
- Type - timeToReach: Float

Example: To move joint 2 by 60 ° and joint 3 by 45 ° in time 5 seconds,

```
apiObj = SvayaApi()
arr = [0.0, 60.0, 45.0, 0.0, 0.0, 0.0]
time = 5.0
apiObj.moveJoints(arr, time)
Or
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 60.0, 45.0, 0.0, 00.0, 0.0], 5.0)
```

4.7.12 moveJointsAsync()

Command: `SvayaApi().moveJointsAsync(jointAngles)`

This command moves the joints to the specified degrees provided by the user. It starts the execution asynchronously, meaning it doesn't wait for the movement to complete before allowing other tasks to run.

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6

Example: To move joint 1 by 90 and joint 3 by 90,

```
apiObj = SvayaApi()
arr = [90.0, 0.0, 90.0, 0.0, 0.0]
apiObj.moveJointsAsync(arr)
```

4.7.13 moveJointsAsync()

Command: `SvayaApi().moveJointsAsync(jointAngles, jointVel, jointAcc)`

This command moves the joints to the specified degrees provided by the user, with the given joint velocity and joint acceleration. It starts execution asynchronously.

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6
- Param - jointVel: Number of degrees per second the joints must move
- Type - jointVel: Float, specifies degrees per second
- Param - jointAcc: Number of degrees per second² the joints must accelerate
- Type - jointAcc: Float, specifies degrees per second²

Example: To move joint 1 by 90° and joint 3 by 45° with joint velocity=90°/s and joint acceleration 100°/s²,

```
apiObj = SvayaApi()
arr = [90.0, 0.0, 45.0, 0.0, 0.0, 0.0]
jointVel = 90.0
jointAcc = 100.0
apiObj.moveJointsAsync(arr, jointVel, jointAcc)
Or
apiObj = SvayaApi()
apiObj.moveJointsAsync([90.0, 0.0, 45.0, 0.0, 0.0, 0.0], 90.0, 100.0)
```

4.7.14 moveJointsAsync()

Command: `SvayaApi().moveJointsAsync(jointAngles, timeToReach)`

This command moves the joints to the specified degrees provided by the user and ensures the target is reached within the specified time. It starts execution asynchronously.

- Param - jointAngles: List [joint angles]
- Type - jointAngles: List [floats], length 6
- Param - timeToReach: Number of seconds the joints should take to reach their joint values
- Type - timeToReach: Float

Example: To move joints 1 by 90 ° and 3 by 45 ° in 5 seconds,

```
apiObj = SvayaApi()
apiObj.moveJointsAsync([90.0, 0.0, 45.0, 0.0, 0.0, 0.0], 20.0)
```

4.7.15 moveToCartPose()

Command: `SvayaApi().moveToCartPose(cartPose)`

This command moves the robot to a specific position in cartesian space as defined by the user. The cartPose parameter specifies the target location and orientation, allowing precise control over the robot's position in 3D space.

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Cartesian pose object

Example: To move to position in base frame where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPose(pose)
```

In case of move to cartesian pose using user frame and tool frame,

```
apiObj = SvayaApi()
userFrame = Frame(-50, -50, 0, 0, 0, 0)
toolFrame = Frame(0, 0, 0, 0, 0, 0)
apiObj.setJogUserFrame(userFrame)
apiObj.setJogToolFrame(toolFrame)
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0],
userFrame, toolFrame)
apiObj.moveToCartPose(pose)
```

4.7.16 moveToCartPose()

Command: `SvayaApi().moveToCartPose(cartpose, linearVel, linearAcc)`

This command moves robot to the cartesian position user has given with the specified linear velocity(linearVel) and linear acceleration(linearAcc).

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Cartesian pose object
- Param - linearVel: Linear velocity at which the robot must move
- Type - linearVel: Float, specifies millimetres per second
- Param - linearAcc: Linear acceleration that the robot must achieve
- Type - linearAcc: Float, specifies millimetres per second squared

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation with linear velocity of 100 mm/s and linear acceleration of 250 mm/s,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.movePose(pose, 100.0, 250.0)
```

4.7.17 moveToCartPose()

Command: `SvayaApi().moveToCartPose(cartpose, timeToReach)`

This command moves robot to the cartesian position user has given and will reach the position on the specified time(timeToReach).

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Cartesian pose object
- Param - timeToReach: Number of seconds the joints should take to reach the given position
- Type - timeToReach: Float

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.movePose(pose, 5.0)
```

4.7.18 moveToCartPose()

Command: `SvayaApi().moveToCartPose(position, orientation)`

This command is used to move the robot to a specified cartesian position with a defined orientation. The target position includes the X, Y, and Z coordinates, while the orientation defines the rotation angles (Rx, Ry, Rz). Both the position and orientation must be provided as separate objects.

- Param - position: Cartesian point to which the robot must move
- Type - position: Object with attributes 'x', 'y', and 'z'
- Param - orientation: Orientation with which the robot must reach the point
- Type - orientation: Object with attributes 'Rx', 'Ry', and 'Rz'

Example: To move to position which has x = 250, y = 50 and z = 100 and orientation with roll = 0, pitch = -180 and yaw = 0,

```
apiObj = SvayaApi()
position = Position([250.0, 50.0, 100.0])
orientation = Orientation([0.0, -180.0, 0.0])
apiObj.moveToCartPose(position, orientation)
```

4.7.19 moveToCartPose()

Command: `SvayaApi().moveToCartPose(position, orientation, linearVel, linearAcc)`

This command is used to move the robot to a specified cartesian position with a given orientation, using specified linear velocity and acceleration. The position defines the X, Y, and Z coordinates and orientation.

- Param - position: Cartesian point to which the robot must move
- Type - position: Object with attributes 'x', 'y', and 'z'
- Param - linearVel: Linear velocity at which the robot must move
- Type - linearVel: Float, specifies millimetres per second
- Param - linearAcc: Linear acceleration that the robot must achieve
- Type - linearAcc: Float, specifies millimetres per second squared

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation with linear velocity of 100 mm/s and linear acceleration of 250 mm/s,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPose(pose, 100.0, 250.0)
```

4.7.20 moveToCartPose()

Command: `SvayaApi().moveToCartPose(position, orientation, timeToReach)`

This command moves robot to given cartesian point with orientation and will reach the position on the specified time (timeToReach).

- Param - position: Cartesian point to which the robot must move
- Type - position: Object with attributes 'x', 'y', and 'z'
- Param - orientation: Orientation with which the robot must reach the point
- Type - orientation: Object with attributes 'x', 'y', and 'z'
- Param - timeToReach: Number of seconds the joints should take to reach the point with the given orientation.
- Type - timeToReach: Float

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation in 5 secs,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPose(pose, 5.0)
```

4.7.21 moveToCartPoseAsync()

Command: `SvayaApi().moveToCartPoseAsync(position,orientation)`

This command is used to move the robot to a specified cartesian position with a given orientation. The motion starts execution asynchronously, meaning it doesn't wait for the movement to complete and allows other tasks to run at the same time.

- Param - position: Cartesian point to which the robot must move
- Type - position: Object of position with attributes 'x', 'y', and 'z'
- Param - orientation: Orientation with which the robot must reach the position
- Type - orientation: Object of orientation with attributes 'x', 'y', and 'z'

Example: To move to position which has x = 250, y = 50 and z = 100 and orientation with roll = 0, pitch = -180 and yaw = 0,

```
apiObj = SvayaApi()
point = Position([250.0, 50.0, 0.0])
orientation = Orientation(0.0, -180.0, 0.0)
apiObj.moveToCartPoseAsync(point, orientaion)
```

4.7.22 moveToCartPoseAsync()

Command: `SvayaApi().moveToCartPoseAsync(position, orientation, linearVel, linearAcc)`

This command moves robot asynchronously to the given cartesian point with orientation with specified linear velocity and linear acceleration.

- Param - position: Cartesian point to which the robot must move
- Type - position: Object of position with attributes 'x', 'y', and 'z'
- Param - orientation: Orientation with which the robot must reach the position
- Type - orientation: Object of orientation with attributes 'x', 'y', and 'z'
- Param - linearVel: Linear velocity at which the robot must move
- Type -linearVel: Float, specifies millimeters per second
- Param - linearAcc: Linear acceleration that the robot must achieve

- Type -linearAcc: Float, specified millimeters per second squared

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation with linear velocity of 100 mm/s and linear acceleration of 250 mm/s,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPoseAsync(pose, 100.0, 250.0)
```

4.7.23 moveToCartPoseAsync()

Command: `SvayaApi().moveToCartPoseAsync(position, orientation, timeToReach)`

This command is used to move the robot to a specified cartesian position with a given orientation, and it will reach the target position within the specified time. The motion starts execution asynchronously.

- Param - position: Cartesian point to which the robot must move
- Type - position: Object of position with attributes 'x', 'y', and 'z'
- Param - orientation: Orientation with which the robot must reach the position
- Type - orientation: Object of orientation with attributes 'x', 'y', and 'z'
- Param - timeToReach: Number of seconds the joints should take to reach the point with the given orientation
- Type - timeToReach: Float

Example: To move to position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation in 5 secs,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPose(pose, 5.0)
```

4.7.24 moveToCartPoseAync()

Command: `SvayaApi().moveToCartPoseAync(cartPose)`

This command is used to move the robot to a specified cartesian pose. The pose includes both position and orientation. The motion starts execution asynchronously.

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Object of cartesian pose

Example: To move to position which have x = 250, y = 50 and z = 100 and orientation with roll = 0, pitch = -180 and yaw = 0,

```
apiObj = SvayaApi()
cartPose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPoseAync(cartPose)
```

4.7.25 moveToCartPoseAsync()

Command: `SvayaApi().moveToCartPoseAsync(cartpose, timeToReach)`

This command is used to move the robot to a specified cartesian position, and it will reach the target point within the given time. The motion starts execution asynchronously.

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Cartesian Pose object
- Param - timeToReach: Number of seconds the joints should take to reach the cartesian position
- Type - timeToReach: Float

Example: To move to position which has x = 250, y = 50 and z = 0 and orientation with roll = 0, pitch = -180 and yaw = 0 in time 5 seconds,

```
apiObj = SvayaApi()
cartPose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPoseAsync(cartPose, 5.0)
```

4.7.26 moveToCartPoseAsync()

Command: `SvayaApi().moveToCartPoseAsync(cartpose, linearVel, linearAcc)`

This command moves the robot to the cartesian position the user has given with the specified joint velocity and joint acceleration, but this command starts execution and doesn't wait for completion, runs asynchronously.

- Param - cartPose: Cartesian pose to which the joints must move
- Type - cartPose: Object of cartesian pose
- Param - linearVel: Linear velocity at which the robot must move
- Type - linearVel: Float, specifies millimeters per second
- Param - linearAcc: Linear acceleration that the robot must achieve
- Type - linearAcc: Float, specified millimeters per second squared

Example: To move to Position where x = 250, y = 50 and z = 100 in cartesian and roll = 0, pitch = -180 and yaw = 0 in orientation with linear velocity of 100 mm/s and linear acceleration of 250 mm/s,

```
apiObj = SvayaApi()
pose = CartesianPose([250.0, 50.0, 100.0, 0.0, -180.0, 0.0])
apiObj.moveToCartPoseAsync(pose, 100.0, 250.0)
```

4.7.27 moveTrajectory()

Command: `SvayaApi().moveTrajectory(poses)`

This command is used to move the robot through a series of positions, called waypoints. Each position is defined using a cartesian pose object. When this function is called, the robot follows the path through all the waypoints using the default speed and acceleration. This creates a continuous and smooth motion.

The main difference is that linear motion only moves the robot from one point to another in a straight line, while command lets the robot move through multiple points smoothly. This is useful when the robot needs to follow a curved or complex path instead of just going straight from one position to another.

- Param - List of Poses: Position objects that must be executed
- Type - List of Poses: List of cartesian pose objects

Example:

```
apiObj = SvayaApi()
pose1 = CartesianPose([250.0, 50.0, 300.0, 0.0, -180.0, 0.0])
pose2 = CartesianPose([350.0, 100.0, 300.0, 0.0, -180.0, 0.0])
pose2 = CartesianPose([150.0, 150.0, 300.0, 0.0, -180.0, 0.0])
arr = [pose1, pose2, pose3]
apiObj.moveTrajectory(arr)
```

4.7.28 moveTrajectory()**Command: SvayaApi().moveTrajectory(poses, velocity, acceleration)**

This command is used to move the robot smoothly through a series of positions, called waypoints. Each position is defined using a cartesian pose object. The robot follows the path through all the waypoints using the user specified velocity and acceleration, resulting in a continuous motion.

- Param - List of Poses: Position objects that must be executed
- Type - List of Poses: List of cartesian pose objects
- Param - velocity: Velocity at which the trajectory should move
- Type - velocity: Float, for cartesian motion, specifies millimetres per second, range [1, 1200] mm/sec
- Param - acceleration: Acceleration at which the trajectory should move
- Type - acceleration: Float, for cartesian motion, specifies millimetres per second squared, range [1, 1800] mm/sec²

Example:

```
apiObj = SvayaApi()
pose1 = CartesianPose([250.0, 50.0, 300.0, 0.0, -180.0, 0.0])
pose2 = CartesianPose([350.0, 100.0, 300.0, 0.0, -180.0, 0.0])
pose2 = CartesianPose([150.0, 150.0, 300.0, 0.0, -180.0, 0.0])
arr = [pose1, pose2, pose3]
tcp_vel = 300
tcp_acceleration = 600
apiObj.moveTrajectory(arr, tcp_vel, tcp_acceleration)
```

4.7.29 moveTrajectoryAsync()**Command: SvayaApi().moveTrajectoryAsync(poses)**

This command is used to move the robot through a series of waypoints, using an asynchronous trajectory. Each position is defined using a cartesian pose object. Unlike synchronous commands, this method starts execution without waiting for the motion to complete, allowing other tasks to run at the same time.

- Param - poses: Position objects that must be executed
- Type - poses: List of cartesian pose objects

Example:

```
apiObj = SvayaApi()
pose1 = CartesianPose([250.0, 50.0, 300.0, 0.0, -180.0, 0.0])
pose2 = CartesianPose([350.0, 100.0, 300.0, 0.0, -180.0, 0.0])
```

```
pose2 = CartesianPose([150.0, 150.0, 300.0, 0.0, -180.0, 0.0])
arr = [pose1, pose2, pos3]
apiObj.moveTrajectoryAsync(arr)
```

4.7.30 moveTrajectoryAsync()

Command: `SvayaApi().moveTrajectoryAsync(poses, velocity, acceleration)`

This command is used to send a predefined trajectory to the robot for execution in an asynchronous manner. The trajectory is defined using a trajectory class object, which includes waypoints using the user specified velocity and acceleration.

- Param - List of Poses: Position objects that must be executed
- Type - List of Poses: List of cartesian pose objects
- Param - velocity: Velocity at which the trajectory should move
- Type - velocity: Float, for cartesian motion, specifies millimetres per second, range [1, 1200] mm/sec
- Param - acceleration: Acceleration at which the trajectory should move
- Type - acceleration: Float, for cartesian motion, specifies millimetres per second squared, range [1, 1800] mm/sec²

Example:

```
apiObj = SvayaApi()
apiObj.moveJoints([90.0, 0.0, 90.0, 0.0, 90.0, 0.0], 45.0, 120.0)
cart_pose = apiObj.getCartPose()
print("Cart Pose X, Y, Z, Roll, Pitch, Yaw", pose.x, pose.y, pose.z,
      pose.roll, pose.pitch, pose.yaw)
pose_1 = CartesianPose([pose.x + 150, pose.y, pose.z - 100,
                        pose.roll, pose.pitch, pose.yaw])
pose_2 = CartesianPose([pose.x + 150, pose.y + 150, pose.z - 100,
                        pose.roll, pose.pitch, pose.yaw])
pose_3 = CartesianPose([pose.x, pose.y + 150, pose.z - 100,
                        pose.roll, pose.pitch, pose.yaw])
pose_4 = CartesianPose([pose.x, pose.y, pose.z - 100, pose.roll,
                        pose.pitch, pose.yaw])
rectangle_sequence = [pose_1, pose_2, pose_3, pose_4, pose_1]
vel = 250.0
acc = 500.0
apiObj.moveTrajectoryAsync(rectangle_sequence, vel, acc)
apiObj.waitForTime(1)
print("Joint velocities are:", apiObj.getJointVelocities())
```

4.7.31 moveTrajectoryInCircle()

Command: `SvayaApi().moveTrajectoryInCircle(poses)`

This command is used to move the robot in a circular path by following a set of 3 waypoints. The robot runs a trajectory through these three positions using the default velocity and acceleration. The three points are used to define the arc of the circle, and the robot follows this arc smoothly to create circular motion.

- Param - ListofPoses: Position objects that must be executed

- Type - ListofPoses: List of cartesian pose objects

Example:

```
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 0.0, 90.0, 0.0, 90.0, 0.0])
cart_pose = apiObj.getCartPose()
print("Cart Pose X, Y, Z, Roll, Pitch, Yaw: ", pose.x, pose.y,
      pose.z, pose.roll, pose.pitch, pose.yaw)
p1 = CartesianPose([pose.x, pose.y, pose.z , pose.roll, pose.pitch,
                    pose.yaw])
p2 = CartesianPose([pose.x + 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
p3 = CartesianPose([pose.x - 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
sequence = [p1, p2, p3]
apiObj.moveTrajectoryInCircle(sequence)
```

 **Warning:** Must provide more than 3 points to define the arc. Incorrect formatting may result in invalid path execution.

4.7.32 moveTrajectoryInCircle()

Command: SvayaApi().moveTrajectoryInCircle(poses, velocity, acceleration)

This command is used to move the robot in a circular path by following a set of 3 waypoints. The robot runs a trajectory through these three positions using the user defined velocity and acceleration. The three points are used to define the arc of the circle, to create circular motion.

- Param - ListofPoses: Position objects that must be executed
- Type - ListofPoses: List of cartesian pose objects, length 3
- Param - velocity: Velocity at which the trajectory should move
- Type - velocity: Float. For cartesian motion, the value is in mm/sec (range [1, 1200])
- Param - acceleration: Acceleration at which the trajectory should move
- Type - acceleration: Float. For cartesian motion, the value is in mm/sec² (range [1, 1800])

Example:

```
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 0.0, 90.0, 0.0, 90.0, 0.0])
cart_pose = apiObj.getCartPose()
print("Cart Pose X, Y, Z, Roll, Pitch, Yaw: ", pose.x, pose.y,
      pose.z, pose.roll, pose.pitch, pose.yaw)
p1 = CartesianPose([pose.x, pose.y, pose.z , pose.roll, pose.pitch,
                    pose.yaw])
p2 = CartesianPose([pose.x + 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
p3 = CartesianPose([pose.x - 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
sequence = [p1, p2, p3]
apiObj.moveTrajectoryInCircle(sequence, 50.0, 50.0)
```

4.7.33 moveTrajectoryInCircleAsync()

Command: SvayaApi().moveTrajectoryInCircleAsync(poses)

This command is used to move the robot in a circular path by following a set of 3 waypoints. The trajectory is executed using the default velocity and acceleration. The method runs asynchronously, meaning it starts the motion immediately without waiting for it to finish, allowing other tasks to run in parallel

- Param - ListofPoses: Position objects that must be executed
- Type - ListofPoses: List of cartesian pose objects or list of joint angles, length 3

Example:

```
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 0.0, 90.0, 0.0, 90.0, 0.0])
cart_pose = apiObj.getCartPose()
print("Cart Pose X, Y, Z, Roll, Pitch, Yaw: ", pose.x, pose.y,
      pose.z, pose.roll, pose.pitch, pose.yaw)
p1 = CartesianPose([pose.x, pose.y, pose.z , pose.roll, pose.pitch,
                    pose.yaw])
p2 = CartesianPose([pose.x + 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
p3 = CartesianPose([pose.x - 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
sequence = [p1, p2, p3]
apiObj.moveTrajectoryInCircleAsync(sequence)
```

4.7.34 moveTrajectoryInCircleAsync()

Command: SvayaApi().moveTrajectoryInCircleAsync(poses, velocity, acceleration)

This command is used to move the robot in a circular path by following a set of 3 waypoints. The trajectory is executed using the user defined velocity and acceleration. The method runs asynchronously.

- Param - ListofPoses: Position objects that must be executed
- Type - ListofPoses: List of cartesian pose objects, length 3
- Param - velocity: Velocity at which the trajectory should move
- Type - velocity: Float. For cartesian motion, the value is in mm/sec (range [1, 1200])
- Param - acceleration: Acceleration at which the trajectory should move.
- Type - acceleration: Float. For cartesian motion, the value is in mm/sec² (range [1, 1800])

Example:

```
apiObj = SvayaApi()
apiObj.moveJoints([0.0, 0.0, 90.0, 0.0, 90.0, 0.0])
cart_pose = apiObj.getCartPose()
print("Cart Pose X, Y, Z, Roll, Pitch, Yaw: ", pose.x, pose.y,
      pose.z, pose.roll, pose.pitch, pose.yaw)
p1 = CartesianPose([pose.x, pose.y, pose.z , pose.roll, pose.pitch,
                    pose.yaw])
p2 = CartesianPose([pose.x + 120, pose.y - 100, pose.z , pose.roll,
                    pose.pitch, pose.yaw])
p3 = CartesianPose([pose.x - 120, pose.y - 100, pose.z , pose.roll,
```

```
pose.pitch, pose.yaw])  
sequence = [p1, p2, p3]  
apiObj.moveTrajectoryInCircleAsync(sequence, 50.0, 50.0)
```

4.8 Basic status and I/O

These commands allow real-time monitoring of the robot's status and access to digital I/O and system feedback. They are useful for diagnostics, safety checks, and integration with external systems (like buttons, sensors, or PLCs).

4.8.1 getState()

Command: `SvayaApi().getState()`

This command returns the current state of the robot.

- Return: State of robot ("ESTOP", "INIT", "SWITCH_ON", "READY", "ENABLE", "REAL", "FAULT", "GRAVITY")
- R-Type: String

4.8.2 getGreenButton()

Command: `SvayaApi().getGreenButton()`

This command returns the status of green button.

- Return: Green button
- R-Type: Boolean

4.8.3 getRedButton()

Command: `SvayaApi().getRedButton()`

This command returns the status of red button.

- Return: Red button
- R-Type: Boolean

4.8.4 getJointErrorsDesc()

Command: `SvayaApi().getJointErrorDesc()`

This command returns joints error description list.

- Return: List of joints error description
- R-Type: List [string]

4.8.5 getDigitalInput()

Command: `SvayaApi().getDigitalInput(id)`

This command returns the digital input status of the requested pin ID.

- Param - id: Digital input pin ID number
- Type - id: Int, range [1, 16] (both 1 and 16 included)
- Return: Digital input state of the given pin
- R-Type: Boolean (True or False)

Example: To get digital Input 1,

```
apiObj = SvayaApi()
digitalInput1 = apiObj.getDigitalInput(1)
```

4.8.6 getDigitalInputs()

Commands: `SvayaApi().getDigitalInputs()`

This command returns the list which contains digital inputs. Access status of digital inputs with index (0 based indices).

- Return: List of digital input states
- R-Type: List [Boolean]

Example: To get digital input 1 from all digital inputs,

```
apiObj = SvayaApi()
arr = apiObj.getDigitalInputs()
digitalInput1 = arr[0]
```

4.8.7 getDigitalOutput()

Commands: `SvayaApi().getDigitalOutput(pinid)`

This command returns the digital output status of requested pin ID.

- Param - id: Digital output pin ID number
- Type - id: Int, range [1, 16] (both 1 and 16 included)
- Return: A list of length 2 containing:
 - [0] Digital output value (True for ON, False for OFF)
 - [1] Output Type (True for NPN, False for PNP)
- R-Type: List [Boolean, Boolean]

Example: To get digital output 1,

```
apiObj = SvayaApi()
digitalOutput1 = apiObj.getDigitalOutput(1)
digitalOutputValue = digitalOutput1[0]
digitalOutputType = digitalOutput1[1]
```

4.8.8 getDigitalOutputs()

Commands: `SvayaApi().getDigitalOutputs()`

This command returns the list of lists, where each list contains digital output's value and type.

- Return: List of digital outputs. Each item is a sub list containing the output value and type
- R-Type: List [list [Boolean]]

Example: To get digital input 1,

```
apiObj = SvayaApi()
arr = apiObj.getDigitalOutputs()
digitalOutput1 = arr[0][0]
```


4.8.9 getAnalogInput()

Command: `SvayaApi().getanalogInput(pinId)`

This command returns the analog input value and type of requested pin ID. In case of type, 0 implies voltage, 1 implies current and 2 implies differential.

- Param - pinId: Analog input pin ID number
- Type - pinId: Int, range [1, 4] (both 1 and 4 included)
- Return: Analog input of the given pin. The returned list has two elements:
 - [0] The analog value (e.g., 5.66)
 - [1] The input type (e.g., 0 for voltage, 1 for current)
- R-Type: List [float, int]

Example: To get analog Input 1 value,

```
apiObj = SvayaApi()
analogInput1 = apiObj.getanalogInput(1)
print(analogInput1)
print(f"analog input 1 value {analogInput1[0]} and Type {analogInput1[1]}")
```

Output: [1.286879,0] analog input 1 value 1.286879 and Type 0

4.8.10 getAnalogInputs()

Command: `SvayaApi().getanalogInputs()`

This command returns the 2D list where each list contains analog input value and type. Access values and types of analog inputs with index (0 based indices). In case of type, 0 implies voltage, 1 implies current and 2 implies differential.

- Return: List of analog inputs. Each sub list contains [value, type], where value is a float and type is an int
- R-Type: List [list [float, int]]

Example: To get analog input 1 value,

```
apiObj = SvayaApi()
arr = apiObj.getanalogInputs()
analogInput1 = arr[0][0]
```

4.8.11 getAnalogOutput()

Command: `SvayaApi().getAnalogOutput(pinId)`

This command returns the analog output value and type of requested pin ID. In the case of type, 0 implies voltage, 1 implies current and 2 implies differential.

- Param - pinId: Analog output pin ID number
- Type - pinId: Int, range [1, 4] (both 1 and 4 included)
- Return: Analog output of the given pin. The return list has two elements:
 - [0] The analog value (float, e.g., 5.66)

- [1] The output type (int, e.g., 0 for voltage, 1 for current)
- R-Type: List [float, int]

Example: To get analog output 1 value,

```
apiObj = SvayaApi()
analogOutput1 = apiObj.getAnalogOutput(1)
print(analogOutput1)
print(f"analog output 1 value {analogOutput1[0]} and Type {analogOutput1[1]}")
```

Output: [1.286879,0] analog output 1 value 1.286879 and Type 0}

4.8.12 getAnalogOutputs()

Command: `SvayaApi().getAnalogOutputs()`

This command returns to the 2D list which contains a list of analog output values and type. Access values and types of analog inputs with index (0 based indices). In the case of type, 0 implies voltage, 1 implies current and 2 implies differential.

- Return: List of analog outputs. Each item is a sub list containing two elements: [0] Analog value (float), [1] Output type (int).
- R-Type: List [list [float, int]]

Example: To get analog output 1 value,

```
apiObj = SvayaApi()
arr = apiObj.getAnalogOutputs()
analogOutput1 = arr[0][0]
```

4.8.13 getToolDigitalInput()

Command: `SvayaApi().getToolDigitalInput(pinId)`

This command returns the tool's digital input status of requested pin ID.

- Param - pinId: Tool's digital input pin ID number
- Type - pinId: Int, range [1, 2] (both 1 and 2 included)
- Return: Tool's digital input state of the given pin
- R-Type: Boolean

Example: To get tool's digital Input 1,

```
apiObj = SvayaApi()
toolDigitalInput1 = apiObj.getToolDigitalInput(1)
```

4.8.14 getToolDigitalInputs()

Command: `SvayaApi().getToolDigitalInputs()`

This command returns the list which contains tool's digital inputs. Access status of tool's digital inputs with index (0 based indices).

- Return: List of tool's digital inputs

- R-Type: List [Boolean]

Example: To get digital input 1,

```
apiObj = SvayaApi()
tool_dig_inputs = apiObj.getToolDigitalInputs()
print(f"Tool digital input 1 is {tool_dig_inputs[0]} and tool digital
input 2 is {tool_dig_inputs[1]}")
```

4.8.15 getToolDigitalOutput()

Command: `SvayaApi().getToolDigitalOutput()`

This command returns the tool's digital output status of requested pin ID.

- Param - pinId: Tool's digital output pin ID number.
- Type - pinId: Int, range [1, 2] (both 1 and 2 included)
- Return: A list containing the tool's digital output value and type of the given pin.
- The list contains:
 - [0] The output value (True/False).
 - [1] The output Type (0 for PNP, 1 for NPN, 2 for HIGH).
- R-Type: List [Boolean, int]

Example: To get tool's digital output 1,

```
apiObj = SvayaApi()
toolDigitalOutput1 = apiObj.getToolDigitalOutput(1)
```

4.8.16 getToolDigitalOutputs()

Command: `SvayaApi().getToolDigitalOutputs()`

This command returns the list of lists, where each list contains tool's digital outputs. Access status of tool's digital outputs with index (0 based indices). 0 index contains value and 1 index contains type PNP/NPN and HIGH.

- Param - pinId: Tool's digital output pin ID number
- Type - pinId: Int, range [1, 2] (both 1 and 2 included)
- Return: List of Tool's digital outputs and their types
- Each sublist contains:
 - [0] The digital output value (True/False)
 - [1] The output Type (0 for PNP, 1 for NPN, 2 for HIGH)
- R-Type: List [list [Boolean, int]]

Example: To get digital output 1,

```
apiObj = SvayaApi()
arr = apiObj.getToolDigitalOutputs()
print(f"Tool digital output 1 is {tool_dig_outputs[0]} and tool
digital output 2 is {tool_dig_outputs[1]}")
```

4.8.17 getToolAnalogInput()

Command: SvayaApi().getToolAnalogInput()

This command returns the list which contains tool's analog input values and type. There is only one tool analog input. First index is value of input, and second index is type. In case of type, 0 implies voltage, 1 implies current and 2 implies differential.

- Return: List of tool's analog inputs
- Each item in the list contains:
 - [0] The analog input value (float)
 - [1] The input Type (int)
- R-Type: List [list [float, int]]

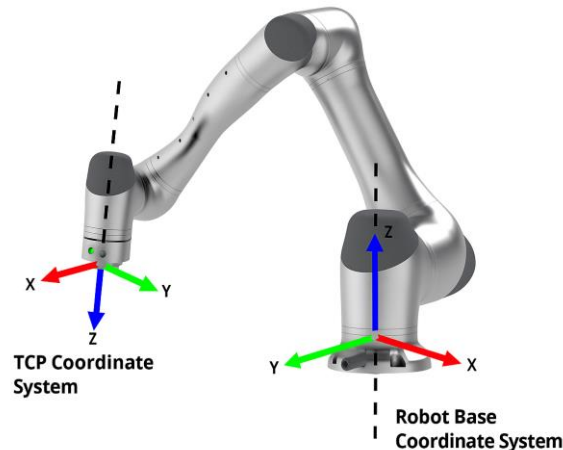
Example: To get tool's analog input value,

```
apiObj = SvayaApi()
arr = apiObj.getToolAnalogInput()
toolAnalogInput = arr[0]
TypeOfAnalogInput = arr[1]
print(toolAnalogInput)
print(TypeOfAnalogInput)
```

Output: 1.653, 0

4.9 Jogging

Jogging refers to the manual movement and fine-tuning of the robot's position. These functions allow an operator to move the robot step-by-step along specific axes (X, Y, Z) in cartesian space. This is useful for tasks like positioning, calibration or making small adjustments during setup. The movement can be controlled by specifying the direction, using predefined values (e.g., 1 for positive direction, -1 for negative direction).



The above figure shows the Robot Base Coordinate System and TCP Coordinate System.

⚠ Warning: Do not perform jogging when the robot is in automatic mode. Jogging is intended only for manual mode.

4.9.1 jog()

Command: `SvayaApi().jog(index, direction)`

This command is used to move a specific joint of a robot in a particular direction. It takes two arguments: the joint number, which indicates which joint to move, and the direction, which specifies the direction of the movement. This command allows the user to manually control a joint by selecting both which joints to move and which direction to move it.

1: for jogging in positive direction

-1: for jogging in negative direction

- Param - index: Joint number which must be jogged
- Type - index: Int, range [1, 6] (both inclusive)
- Param - direction: Direction in which jogging must be done; +ve and -ve which are 1 and -1 respectively
- Type - direction: Int, allowed values are 1 and -1

Example: To start jogging of 1st joint in positive direction,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj.jog(1,1)
```

```
waitForTime(3)
apiObj.stopCartesianJogging()
```

 **Note:** Always set jog speed using **setJogSpeed()** before jogging.

 **Caution:** Jog commands continue movement until explicitly stopped. Use **stopCartesianJogging()** appropriately after **waitForTime()**.

4.9.2 jogForTime()

Command: **SvayaApi().jogForTime(index, direction, time)**

This command is used to move a specific joint of a robot in a chosen direction for a certain amount of time. It takes three inputs: the joint number, which joint to move; the direction, whether the joint should move in positive or negative direction and the time in seconds.

1: for jogging in positive direction

-1: for jogging in negative direction

- Param - index: Joint number which must be jogged
- Type - index: Int, range [1, 6]
- Param - direction: Direction in which jogging must be done
- Type - direction: Int (1 or -1)
- Param - time: Time duration of the jogging in seconds
- Type - time: Int or float

Example: To start jogging of 1st joint in positive direction for 5 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj.jogForTime(1, 1, 5)
```

 **Caution:** Do not exceed safe durations or speeds. Use only in manual mode.

4.9.3 jogAlong_X()

Command: **SvayaApi().jogAlong_X(direction)**

This command is used to move the robot along the X-axis in cartesian space. By calling, you can jog the robot either positive or negative along the X-axis.

- Param - direction: Direction in which the robot should jog along the X-axis. Use 1 for positive (+X) direction and -1 for negative (-X) direction
- Type - direction: Int (1 or -1)

Example: To start jogging along X-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj.jogAlong_X(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.4 jogAlong_Y()

Command: `SvayaApi().jogAlong_Y(direction)`

This command is used to move the robot along the Y-axis in cartesian space. By calling, the robot can be jogged either in the positive or negative Y direction.

- Param - direction: Direction in which the robot should jog along the Y-axis. Use 1 for positive (+Y) direction and -1 for negative (-Y) direction
- Type - direction: Int (1 or -1)

Example: To start jogging along Y-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj.jogAlong_Y(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.5 jogAlong_Z()

Command: `SvayaApi().jogAlong_Z(direction)`

The method is used to move the robot along the Z-axis in cartesian space. By calling, the robot can be jogged either upward or downward along the Z-axis. This function is useful for manually adjusting the robot's vertical position.

- Param - direction: Direction in which the robot should jog along the Z-axis. Use 1 for positive (+Z) direction and -1 for negative (-Z) direction
- Type - direction: Int (1 or -1)

Example: To start jogging along Z-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj.jogAlong_Z(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.6 rotateAlong_X()

Command: `SvayaApi().rotateAlong_X(direction)`

The command rotates the robot's Tool Centre Point (TCP) around the X-axis. The direction Parameter determines the rotation direction: use 1 for positive rotation and -1 for negative rotation.

- Param - direction: Direction in which the robot should jog along the X-axis. Use 1 for positive (+X) direction and -1 for negative (-X) direction
- Type - direction: Int (1 or -1)

Example: To start rotating along X-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
```

```
apiObj. rotateAlong_X(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.7 rotateAlong_Y()

Command: `SvayaApi().rotateAlong_Y(direction)`

The command rotates the robot's TCP around the Y-axis.

- Param - direction: Direction in which the robot should jog along the Y-axis. Use 1 for positive (+Y) direction and -1 for negative (-Y) direction
- Type - direction: Int (1 or -1)

Example: To start rotating along Y-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj. rotateAlong_Y(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.8 rotateAlong_Z()

Command: `SvayaApi().rotateAlong_Z(direction)`

The command rotates the robot's TCP around the Z-axis.

- Param - direction: Direction in which the robot should jog along the Z-axis. Use 1 for positive (+Z) direction and -1 for negative (-Z) direction
- Type - direction: Int (1 or -1)

Example: To start rotating along Z-axis in positive direction for 3 seconds,

```
apiObj = SvayaApi()
apiObj.setJogSpeed(30)
apiObj. rotateAlong_Z(1)
waitForTime(3)
apiObj.stopCartesianJogging()
```

4.9.9 jointStepJog()

Command: `SvayaApi().jointStepJog(index, direction)`

The command jogs a specific joint by a small step, allowing fine-tuned manual adjustments. It takes two parameters: index, which specifies the joint number to move, and direction, which indicates the direction of movement use 1 for positive and -1 for negative. The step size is defined by the value set using **setJointJogStep**; if not set, it defaults to 1 degree.

1: for jogging in positive direction

-1: for jogging in negative direction

- Param - index: Joint number that must be jogged. Valid values range from 1 to 6 (inclusive)
- Type - index: Int

- Param - direction: Direction in which jogging must be done. Use 1 for positive direction and -1 for negative direction
- Type - direction: Int

 **Do:** Set the step size beforehand using **setJointJogStep()** if required.

4.9.10 stepJogAlong_X()

Command: **SvayaApi().stepJogAlong_X(direction)**

This command is used to jog the robot along the X-axis by a fixed distance set using **setCartesianJogStep**.

- Param - direction: Direction in which the robot should jog
- Type - direction: Int, 1 for +X direction and -1 for -X direction

4.9.11 stepJogAlong_Y()

Command: **SvayaApi().stepJogAlong_Y(direction)**

This command is used to jog the robot along the Y-axis by a fixed distance set using **setCartesianJogStep**.

- Param - direction: Direction in which the robot should jog
- Type - direction: Int, 1 for +Y direction and -1 for -Y direction

4.9.12 stepJogAlong_Z()

Command: **SvayaApi().stepJogAlong_Z(direction)**

This command is used to jog the robot along the Y-axis by a fixed distance set using **setCartesianJogStep**.

- Param - direction: Direction in which the robot should jog
- Type - direction: Int, 1 for +Y direction and -1 for -Y direction

4.9.13 stepRotateAlong_X()

Command: **SvayaApi().stepRotateAlong_X(direction)**

This command is used to rotate the robot along the X-axis by a fixed angle set using **setJointJogStep**.

- Param - direction: Direction in which the robot should rotate.
- Type - direction: Int, 1 for clockwise (+X) and -1 for counter-clockwise (-X)

4.9.14 stepRotateAlong_Y()

Command: **SvayaApi().stepRotateAlong_Y(direction)**

This command is used to rotate the robot along the Y-axis by a fixed angle set using **setJointJogStep**.

- Param - direction: Direction in which the robot should rotate
- Type - direction: Int, 1 for clockwise (+Y) and -1 for counter-clockwise (-Y)

4.9.15 stepRotateAlong_Z()

Command: **SvayaApi().stepRotateAlong_Z(direction)**

This command is used to rotate the robot along the Z-axis by a fixed angle set using **setJointJogStep**.

- Param - direction: Direction in which the robot should rotate
- Type - direction: Int, 1 for clockwise (+Z) and -1 for counter-clockwise (−Z)

4.10 Emergency and stop function

These commands are used to stop the robot or halt ongoing jogging operations. They are useful in situations where immediate stopping is required for safety or control. These functions do not Return any value and execute immediately when called.

4.10.1 stopRobot()

Command: `SvayaApi().stopRobot()`

This command is used to immediately stop the robot. It halts all current motions regardless of the operation in progress.

- Return: No return value

4.10.2 stopJointJogging()

Command: `SvayaApi().stopJointJogging()`

This command stops the ongoing joint jogging of the robot. It is used when the robot is being moved manually by controlling individual joints.

- Return: No return value

 **Caution:** Only applicable when the robot is being controlled in joint jogging mode.

4.10.3 stopCartesianJogging()

Command: `SvayaApi().stopCartesianJogging()`

This command stops the ongoing cartesian jogging. Cartesian jogging involves moving the robot's TCP in space along X, Y, Z or rotational axes.

- Return: No return value

 **Caution:** Only applicable when the robot is being controlled in cartesian jogging mode.

The following chapter shows example applications for the Svaya robot. The examples showcased in the next chapter demonstrate Svaya API capabilities and some of the possibilities of using it with Python.

5 Example Programs

5.1 Example 1: Robot Basic Functions

1. **Launch the example program:** Open the Python script named API_Reference.py in Integrated Development Environment (IDE).
2. Run the script to verify that the robot is successfully connected to the PC. Upon successful connection, the robot should be enabled, and all brakes should engage automatically.
3. The program made into several tasks, each assigned with a unique number. These tasks represent individual API functionalities. The user is prompted to enter a number corresponding to a specific task they wish to execute.
4. After reviewing the program logic, the user can select and execute any task by entering the task number.

Program:

```
from SvayaAPI.CartesianPose import CartesianPose
from SvayaAPI.clientApi import SvayaApi
from SvayaAPI.Trajectory import Trajectory
import tkinter as tk
import queue

ip = "192.168.1.10"

def error_callack(error_priority, error_msg, error_status):
    print("Error: ", error_priority, error_msg, error_status)
    if error_priority == "info":
        print("Info: ", error_msg, error_status)
    elif error_priority == "medium":
        print("Medium: ", error_msg, error_status)
    elif error_priority == "high":
        print("High: ", error_msg, error_status)

def main():
    try:
        cobot = SvayaApi() #Assigning the variable "cobot" to API
        cobot.initialize(ip, error_callack)
        print("The robot is initialized successfully")
        # cobot.disableRobot()
        cobot.enableRobot() #Makes the cobot enable (disengages the brakes)
        cobot.waitForTime(0.5) #sleeps the execution for a duration of time
        cobot.switchToReal() #switches the robot to real mode
        cobot.waitForTime(0.5)
        cobot.switchToAuto() #switches the robot to automatic mode
        cobot.waitForTime(0.5)

        while True:
            cobot.waitForTime(0.5)
            task_number = int(input("Which task you want to Execute?\n"))
            print("The Entered Task Number is:", task_number)
```

```

if task_number == 0:
    state_of_robot = cobot.getState()
    print("Cobot status is ", state_of_robot)
    joint_errors = cobot.getJointsErrorDesc()
    print("Joint errors are ", joint_errors)

elif task_number == 1:
    analog_input1 = cobot.getAnalogInput(1)
    print("Analog input 1 value : ", analog_input1)
    print(f"Analog input 1 value {analog_input1[0]} and mode
{analog_input1[1]} ")
    arr = cobot.getAnalogInputs()
    print("analog input 1:", arr[0])
    print("analog input 2:", arr[1])
    print("analog input 3:", arr[2])
    print("analog input 4:", arr[3])

elif task_number == 2:
    analog_output1 = cobot.getAnalogOutput(1)
    print("Analog output 1 value : ", analog_output1)
    print(f"Analog output 1 value {analog_output1[0]} and mode
{analog_output1[1]} ")
    arr = cobot.getAnalogOutputs()
    print("analog output 1:", arr[0])
    print("analog output 2:", arr[1])
    print("analog output 3:", arr[2])
    print("analog output 4:", arr[3])

elif task_number== 3:
    tool_analog_ip = cobot.getToolAnalogInput()
    print("Tool Analog input value is ", tool_analog_ip)

elif task_number == 4:
    cobot.moveJoints([90.0, 0.0, 90.0, 0.0, 0.0, 0.0])
    cart_array = cobot.getCartPoseArray()
    print("Cartesian Position of Cobot : ", cart_array)

elif task_number == 5:
    cobot.moveJoints([0.0, 0.0, 90.0, 0.0, 0.0, 0.0])
    cart_pose = cobot.getCartPose()
    print("X:", cart_pose.x, "mm")
    print("Y:", cart_pose.y, "mm")
    print("Z:", cart_pose.z, "mm")
    print("RX:", cart_pose.roll, "degrees")
    print("RY:", cart_pose.pitch, "degrees")
    print("RZ:", cart_pose.yaw, "degrees")

elif task_number == 6:
    digital_input = cobot.getDigitalInput(1)
    print(cobot.getDigitalInput(1), cobot.getDigitalInputs())
    if digital_input == 1:

```

```

        print("Digital I/P 1 is Working")
        cobot.waitForTime(0.5)
        cobot.moveJoints([0.0, 0.0, 0.0, 0.0, 0.0, 150.0])
        cobot.waitForTime(1)
        cobot.moveJoints([0.0, 0.0, 0.0, 0.0, 0.0, -150.0])

    elif task_number == 7:
        cobot.setDigitalOutput(1,False, True)
        cobot.waitForTime(2)
        digital_output = cobot.getDigitalOutput(1)
        print(cobot.getDigitalOutput(1),cobot.digital_outputs)
        if digital_output[0]== 1:
            print("Digital O/P 1 is Working")
            cobot.waitForTime(0.5)
            cobot.moveJoints([0.0, 0.0, 0.0, 0.0, 0.0, 150.0])
            cobot.waitForTime(1)
            cobot.moveJoints([0.0, 0.0, 0.0, 0.0, 0.0, -150.0])

    elif task_number == 11:
        joint_values = cobot.getJointValues()
        print("Joint values of cobot : ", joint_values)

    else:
        print("\033[31mThe entered Task Number", task_number, "is not
Valid\033[0m") # 31m is for Red Colour

    except KeyboardInterrupt as e:
        print(e)
        print("\033[31m!!! Aborting, Main Program !!! \n\nPlease Wait... \033[0m")
# 31m is for Red Colour
        cobot.__del__()

if __name__ == "__main__":
    main()

```

5.2 Example 2: Control the Robot using an X-box controller

This example helps to control the cobot manually using an Xbox controller. You can move the robot in different directions and control the gripper using joystick movements and button presses.

How it works:

- When the program starts, the robot connects to the given IP address, gets enabled, and moves to a starting position.
- A background thread begins listening to input from the Xbox controller.
- As you move the joysticks or press buttons, the robot responds accordingly.

Joystick Controls:

- Move the left joystick left or right to move the robot along the X-axis.
- Move the left joystick up or down to move along the Y-axis.
- Move the right joystick left or right to move along the Z-axis.

To stop the robot's movement, simply release the joystick.

Gripper Controls:

- Press the left joystick button to close the gripper.
- Press the right joystick button to open the gripper.

Stopping the Program:

To stop the program at any time, press Ctrl + C in the terminal

Program:

```
import time
import threading
import inputs # Install with `pip install inputs`
from SvayaAPI.clientApi import SvayaApi
from SvayaAPI.CartesianPose import CartesianPose

ip = "192.168.1.10" # Cobot IP address
cobot = SvayaApi()
cobot.initialize(ip)
print("The cobot is initialized successfully")
cobot.enableRobot()
cobot.switchToReal()
cobot.waitForTime(1)
cobot.moveJoints([90.0, 0.0, 90.0, 0.0, 85.0, 0.0])
robot_running = False # Global variables
lock = threading.Lock()
# Deadzone threshold to avoid jittering
JOYSTICK_DEADZONE = 8000
def move_cobot(dx, dy, dz):
    global robot_running

    with lock:
```

```

    if robot_running:
        return
    robot_running = True
try:
    if dx != 0:
        cobot.setJogSpeed(15)
        cobot.jogAlong_X(dx)
    elif dy != 0:
        cobot.setJogSpeed(15)
        cobot.jogAlong_Y(dy)
    elif dz != 0:
        cobot.setJogSpeed(15)
        cobot.jogAlong_Z(dz)
    print(f"Robot moved by (dx={dx}, dy={dy}, dz={dz}) units")

except Exception as e:
    print(f"Error while moving the cobot: {e}")
    reset_robot_running()

def reset_robot_running():
    global robot_running
    cobot.stopCartesianJogging()
    with lock:
        robot_running = False
    print("Robot stopped moving")

def xbox_controller_listener():
    while True:
        events = inputs.get_gamepad()
        for event in events:
            if event.ev_type == 'Absolute':
                if event.code == 'ABS_X':
                    # Left joystick left/right movement (X-axis)
                    if event.state < -JOYSTICK_DEADZONE:
                        print("Left joystick left")
                        move_cobot(-1, 0, 0)
                    elif event.state > JOYSTICK_DEADZONE:
                        print("Left joystick right")
                        move_cobot(1, 0, 0)
                    else:
                        reset_robot_running()
                elif event.code == 'ABS_Y':
                    # Left joystick up/down movement (Y-axis)
                    if event.state < -JOYSTICK_DEADZONE:
                        print("Left joystick up")
                        move_cobot(0, 1, 0)
                    elif event.state > JOYSTICK_DEADZONE:
                        print("Left joystick down")
                        move_cobot(0, -1, 0)
                    else:
                        reset_robot_running()
                elif event.code == 'ABS_RX':

```



```

        # Right joystick left/right movement (Z-axis)
        if event.state < -JOYSTICK_DEADZONE:
            print("Right joystick left (Z-down)")
            move_cobot(0, 0, -1)
        elif event.state > JOYSTICK_DEADZONE:
            print("Right joystick right (Z-up)")
            move_cobot(0, 0, 1)
        else:
            reset_robot_running()
    elif event.code == 'ABS_RY':
        # Right joystick up/down - currently unused
        if event.state < -JOYSTICK_DEADZONE:
            print("Right joystick up")
            move_cobot(0, 0, 0)
        elif event.state > JOYSTICK_DEADZONE:
            print("Right joystick down")
            move_cobot(0, 0, 0)
        else:
            reset_robot_running()
    time.sleep(0.01)

while True:
    events = inputs.get_gamepad()
    for event in events:
        if event.ev_type == 'Key':
            # Gripper controls (Left button: close, Right button: open)
            if event.code == 'BTN_THUMBL' and event.state == 1: # Left button
pressed
                print("Left joystick button pressed: Closing gripper")
                cobot.setToolDigitalOutput(2, False)
                cobot.setToolDigitalOutput(1, True)
                cobot.waitForTime(2)
                # Close the gripper
            elif event.code == 'BTN_THUMBR' and event.state == 1: # Right
button pressed
                print("Right joystick button pressed: Opening gripper")
                cobot.setToolDigitalOutput(1, False)
                cobot.setToolDigitalOutput(2, True)
                cobot.waitForTime(2)

            time.sleep(0.1)
def main():
    xbox_thread = threading.Thread(target=xbox_controller_listener)
    xbox_thread.start()

if __name__ == "__main__":
    main()
    try:
        while True:
            time.sleep(1) # Keeps the main thread alive while the Xbox listener
runs
    except KeyboardInterrupt:
        print("Program interrupted. Shutting down...")

```

5.3 Example 3: Record Robot joint and sensor data

This example records the robot's joint positions, joint torque sensor (JTS) data, and force/torque sensor values while moving through a set of predefined joint configurations. The data is collected at regular intervals and saved to a CSV file named with the current date and time. This is useful for monitoring robot behaviour, analysing sensor performance, or storing movement logs for later review.

Program:

```
from SvayaAPI.CartesianPose import CartesianPose
from SvayaAPI.Orientation import Orientation
from SvayaAPI.Position import Position
from SvayaAPI.clientApi import SvayaApi
from SvayaAPI.Trajectory import Trajectory
import time
import csv
from datetime import datetime

current_datetime = datetime.now()
current_date = current_datetime.strftime('%Y-%m-%d') #Format: YYYY-MM-DD
current_time = current_datetime.strftime('%H:%M:%S') # Format: HH:MM:SS
file_name=current_date+"_"+current_time
file_path = f'{file_name}.csv'
ip="192.168.1.10"
cobot = SvayaApi()
cobot.initialize(ip)
cobot.enableRobot()
time.sleep(0.1)
cobot.switchToReal()
time.sleep(1)
cobot.switchToAuto()
time.sleep(2)
# new_data = [["J1", "J2", "J3", "J4", "J5", "J6", "JTS_J1", "JTS_J2", "JTS_J3",
# "JTS_J4", "JTS_J5", "JTS_J6", "F_x", "F_y", "F_z", "T_x", "T_y", "T_z"]]
new_data = [["J1", "J2", "J3", "J4", "J5", "J6", "JTS_J1", "JTS_J2", "JTS_J3",
"JTS_J4", "JTS_J5", "JTS_J6"]]
with open(file_path, 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(new_data)
cobot.moveJoints([0.0,0.0,0.0,0.0,0.0,0.0])
cobot.setFTBiasState(True, 3)
time.sleep(1)
cobot.setFTBiasState(False, 3)
time.sleep(0.5)
count=0
new_data = []
while count<30:
    count=count+1
    joint_pos = cobot.getJointValues()
    jts = cobot.getJTSBiasedData()
    fts_f = cobot.getFTForceData()
```

```

    fts_t = cobot.getFTTorqueData()
    data =
[joint_pos[0],joint_pos[1],joint_pos[2],joint_pos[3],joint_pos[4],joint_pos[5],
    jts[0],jts[1],jts[2],jts[3],jts[4],jts[5],
    fts_f[0],fts_f[1],fts_f[2],
    fts_t[0],fts_t[1],fts_t[2]]
    new_data.append(data)
    time.sleep(0.5)
cobot.moveJoints([0.0,0.0,90.0,0.0,-90.0,0.0])
count=0
while count<30:
    count = count + 1
    joint_pos = cobot.getJointValues()
    jts = cobot.getJTSBiasedData()
    fts_f = cobot.getFTForceData()
    fts_t = cobot.getFTTorqueData()
    data = [joint_pos[0], joint_pos[1], joint_pos[2], joint_pos[3], joint_pos[4],
joint_pos[5],
        jts[0], jts[1], jts[2], jts[3], jts[4], jts[5],
        fts_f[0], fts_f[1], fts_f[2],
        fts_t[0], fts_t[1], fts_t[2]]
    new_data.append(data)
    time.sleep(0.5)
cobot.moveJoints([90.0,0.0,90.0,0.0,-90.0,0.0])
count=0
while count<30:
    count = count + 1
    joint_pos = cobot.getJointValues()
    jts = cobot.getJTSBiasedData()
    fts_f = cobot.getFTForceData()
    fts_t = cobot.getFTTorqueData()
    data = [joint_pos[0], joint_pos[1], joint_pos[2], joint_pos[3], joint_pos[4],
joint_pos[5],
        jts[0], jts[1], jts[2], jts[3], jts[4], jts[5],
        fts_f[0], fts_f[1], fts_f[2],
        fts_t[0], fts_t[1], fts_t[2]]
    new_data.append(data)
    time.sleep(0.5)
# cobot.moveJointsAsync([90.0,0.0,90.0,0.0,-90.0,0.0])
# time.sleep(2)
# cobot.disableRobot()
with open(file_path, 'a', newline='') as csvfile:
    writer = csv.writer(csvfile)
    writer.writerows(new_data)
cobot.__del__()

```

5.4 Example 4: Handling collision fault condition

```

import time
from SvayaAPI.CartesianPose import CartesianPose
from SvayaAPI.Frame import Frame
from SvayaAPI.clientApi import SvayaApi
from SvayaAPI.Trajectory import Trajectory
import tkinter as tk
import queue

ip = "192.168.1.10"

error_flag = False
no_of_errors = 0
main_gen = None

def error_callback(error_msg, error_priority, error_status):
    global error_flag, no_of_errors
    print("Error:", error_priority, error_msg, error_status)

    if error_priority == "info":
        return
    elif error_priority == "medium":
        if error_msg == "Collision detected":
            if error_status == "SET":
                error_flag = True
                no_of_errors += 1
            elif error_status == "CLEAR":
                error_flag = False
                no_of_errors -= 1
    else: # High/Critical
        print("Critical Error! Execution should be halted.")

def main():
    try:
        cobot = SvayaApi()
        cobot.initialize(ip, error_callback)
        cobot.disableRobot()
        cobot.enableRobot()
        cobot.waitForTime(0.5)
        cobot.switchToReal()
        cobot.waitForTime(0.5)
        cobot.switchToAuto()
        cobot.waitForTime(0.5)

        cobot.moveJoints([-90.0, 0.0, 90.0, 0.0, 90.0, 0.0], 45.0, 90.0)
        yield
        cart = cobot.getCartPose()
        pose_1 = CartesianPose([cart.x + 150, cart.y, cart.z - 100, cart.roll,
                                cart.pitch, cart.yaw])
        pose_2 = CartesianPose([cart.x + 150, cart.y + 150, cart.z - 100,

```

```

cart.roll, cart.pitch, cart.yaw])
    pose_3 = CartesianPose([cart.x, cart.y + 150, cart.z - 100, cart.roll,
cart.pitch, cart.yaw])
    pose_4 = CartesianPose([cart.x, cart.y, cart.z - 100, cart.roll,
cart.pitch, cart.yaw])

    for pose in [pose_4, pose_1, pose_2, pose_3, pose_4]:
        cobot.moveToCartPose(pose)
        yield

except KeyboardInterrupt:
    print("\033[31mAborted by user. Cleaning up...\033[0m")
    cobot.__del__()

if __name__ == "__main__":
    print("Starting Program...")
    main_gen = main()
    print("after main start")

    while True:
        if no_of_errors == 0:
            try:
                next(main_gen)
            except StopIteration:
                break
        else:
            print("Pending errors:", no_of_errors)
            time.sleep(0.2)

```